源程序鉴别材料 • 开头连续部分 1/30

```
# ===== transmission_path_planner/__init__.py =====
# transmission_path_planner/__init__.py
"""输电线路启发式规划系统 V5.0

面向固定站点间输电线路走廊初筛，默认使用确定性 A*。
VIN-Grad 与 PPO 仅可选作诊断，真实线路只用于规划后的独立评估。
"""

from .config import *
from . import geometry, io_export, data, osm, cost, methods, smoothing, metrics, stations, visualization, cli
__version__ = "5.0"
__all__ = ["geometry", "io_export", "data", "osm", "cost", "methods",
           "smoothing", "metrics", "stations", "visualization", "cli"]
# ===== transmission_path_planner/__main__.py =====
# transmission_path_planner/__main__.py
"""支持 python -m transmission_path_planner 运行。"""

from .cli import main

if __name__ == "__main__":
    main()
# ===== transmission_path_planner/cli.py =====
# transmission_path_planner/cli.py
"""命令行入口与多走廊批量编排。"""

import hashlib

from .config import *
from .geometry import *
from .data import *
from .osm import *
from .cost import *
from .methods import *
from .smoothing import *
from .metrics import *
from .visualization import *
from .io_export import *
from .stations import *
from .official_hazards import load_official_hazard_masks
from .provenance import capture_run_provenance, write_run_provenance
from .experimental_theta_star import theta_star_cost_path
from .experimental_tower_graph import span_clearance_status, tower_graph_route
from .experimental_astar_16 import (
    NEIGHBORS_8,
    NEIGHBORS_16,
    astar_exact_integral_path,
    integrate_path_cost,
)
from .diverse_routes import sampled_pareto_astar_routes
from shapely import make_valid
```

源程序鉴别材料 • 开头连续部分 2/30

```
FINAL_ROUTE_LABEL = "最终路线"
ASTAR_16_METHOD = "A*(16-neighbor experimental)"
ASTAR_8_EXACT_METHOD = "A*(8-neighbor exact experimental)"
ASTAR_8_SHARED_METHOD = "A*(8-neighbor shared objective experimental)"
ASTAR_16_SHARED_METHOD = "A*(16-neighbor shared objective experimental)"
TOWER_GRAPH_METHOD = "Tower graph (experimental)"


def _select_final_route(methods, case_metrics, select_tower_graph=False):
    if select_tower_graph:
        metrics = case_metrics.get(TOWER_GRAPH_METHOD, {})
        clearance_spans = metrics.get("clearance_profile_spans", [])
        clearance_passed = (
            metrics.get("clearance_profile_status") in {
                "dem_only_profile_screening",
                "station_elevation_anchored_profile_screening",
            }
            and bool(clearance_spans)
            and all(span.get("status") == "sampled_clearance_ok"
                    for span in clearance_spans)
        )
        if (methods.get(TOWER_GRAPH_METHOD)
                and metrics.get("candidate_validation_status") == "survived_validation"
                and metrics.get("hard_constraint_status") == "pass"
                and metrics.get("hard_violations") == 0
                and metrics.get("building_geometry_status") == "pass"
                and metrics.get("tower_span_status") == "pass"
                and metrics.get("tower_span_violation_count") == 0
                and clearance_passed):
            return TOWER_GRAPH_METHOD, None, "explicit_tower_graph_screen_passed"
        return None, None, "no_hard_constraint_compliant_route"

    for method, rule in (
        ("A*(turn-aware)", "turn_penalized_screening_candidate"),
        ("A*(baseline)", "astar_min_cost_priority"),
        ("A*(weighted)", "weighted_astar_only_candidate"),
    ):
        metrics = case_metrics.get(method, {})
        if methods.get(method) and metrics.get("hard_violations") == 0:
            return method, None, rule
    return None, None, "no_hard_constraint_compliant_route"


def _metric_grade_penalty_weight(method, experimental_neighbor_methods,
                                 configured_weight):
    if method in experimental_neighbor_methods:
        return experimental_neighbor_methods[method].get("grade_penalty_weight", 0.0)
    if method in (TOWER_GRAPH_METHOD, "Theta*(experimental)"):
```

源程序鉴别材料 • 开头连续部分 3/30

```
        return 0.0
    return configured_weight


def _method_route_metrics(results, method):
    route_metrics = []
    for result in results:
        metrics = result.get("metrics", {}).get(method)
        if isinstance(metrics, dict) and "length_km" in metrics and "mean_cost" in metrics:
            route_metrics.append(metrics)
    return route_metrics


def attach_station_endpoints(geo, station_task):
    """Replace snapped raster-center endpoints with the supplied station coordinates."""
    if not station_task or not geo:
        return geo, {}
    start, end = station_task["from_station"], station_task["to_station"]
    start_point = (float(start["lon"]), float(start["lat"]))
    end_point = (float(end["lon"]), float(end["lat"]))
    offsets = {
        "from_station_to_grid_center_offset_m": round(haversine_km(*start_point, *geo[0]) * 1000.0, 2),
        "to_station_to_grid_center_offset_m": round(haversine_km(*end_point, *geo[-1]) * 1000.0, 2),
    }
    if len(geo) == 1:
        return [start_point, end_point], offsets
    adjusted = list(geo)
    adjusted[0], adjusted[-1] = start_point, end_point
    return adjusted, offsets


def expand_single_cell_route(geo, start_ll, end_ll):
    """Turn a one-cell route into the short segment between distinct input endpoints."""
    if not geo or len(geo) != 1:
        return geo
    if tuple(start_ll) == tuple(end_ll):
        raise ValueError("起终点坐标相同，不能生成零长度线路")
    return [(start_ll[1], start_ll[0]), (end_ll[1], end_ll[0])]


def attach_input_endpoints(geo, start_ll, end_ll):
    """Restore both exact input endpoints after routing on snapped raster cells."""
    if not geo:
        return geo
    start_xy = (float(start_ll[1]), float(start_ll[0]))
    end_xy = (float(end_ll[1]), float(end_ll[0]))
    if len(geo) == 1:
        if start_xy == end_xy:
            raise ValueError("起终点坐标相同，不能生成零长度线路")
        return [start_xy, end_xy]
```

源程序鉴别材料 • 开头连续部分 4/30

```
    adjusted = list(geo)
    adjusted[0], adjusted[-1] = start_xy, end_xy
    return adjusted


def validate_route_hard_constraints(geo, transform, hard_mask):
    """Reject a candidate whose exported geometry crosses a blocked grid cell."""
    if not geo:
        return None, {"hard_constraint_status": "no_path", "hard_violations": 0}
    violations = path_hard_violations(geo, transform, hard_mask)
    report = {
        "hard_constraint_status": "pass" if violations == 0 else "fail",
        "hard_violations": int(violations),
    }
    return (geo if violations == 0 else None), report


def validate_tower_span_lengths(geo, min_span_m, max_span_m, tolerance_m=0.5):
    """Recheck every final tower span after exact endpoint restoration."""
    if not geo or len(geo) < 2:
        return None, {"tower_span_status": "no_path", "tower_span_violation_count": 0,
                      "tower_span_lengths_m": []}
    lengths = [haversine_km(*first, *second) * 1000.0
               for first, second in zip(geo, geo[1:])]
    violations = [index for index, length in enumerate(lengths)
                  if length < min_span_m - tolerance_m
                  or length > max_span_m + tolerance_m]
    report = {
        "tower_span_status": "pass" if not violations else "fail",
        "tower_span_violation_count": len(violations),
        "tower_span_lengths_m": [round(length, 2) for length in lengths],
        "tower_span_invalid_indices": violations,
        "tower_span_min_m": min_span_m,
        "tower_span_max_m": max_span_m,
        "tower_span_length_method": "haversine_horizontal_distance_after_endpoint_restore",
    }
    return (geo if not violations else None), report


def tower_candidate_validation_status(geo, searched_path, clearance_status,
                                      building_geometry_status):
    if str(clearance_status).startswith("unknown_"):
        return "unknown_clearance_reference"
    if not geo:
        return "rejected_after_search" if searched_path else "no_path"
    if building_geometry_status != "pass":
        return "incomplete_building_geometry_check"
    return "survived_validation"
```

源程序鉴别材料 • 开头连续部分 5/30

```
def overhead_span_grade_report():
    """Mark ground-grade as unassessed for overhead conductor spans."""
    return {
        "grade_limit_status": "not_applicable_overhead_spans",
        "grade_limit_assessed": False,
        "grade_limit_violation_count": None,
        "grade_limit_unassessed_count": None,
        "grade_limit_max_sampled_grade_ratio": None,
        "grade_limit_threshold_ratio": None,
    }


def route_search_window_clearance_m(geo, transform, shape, pixel_spacing):
    """Return the closest route-vertex distance to the raster window edge."""
    if not geo:
        return None
    height, width = shape
    row_m, col_m = map(float, pixel_spacing)
    inverse = ~transform
    clearance = math.inf
    for lon, lat in geo:
        col, row = inverse * (lon, lat)
        clearance = min(
            clearance,
            row * row_m,
            (height - row) * row_m,
            col * col_m,
            (width - col) * col_m,
        )
    return round(float(clearance), 2)


def screening_data_quality(osm_layer_metadata):
    """Describe missing settlement proxy layers without treating them as empty land."""
    required_layers = ("buildings", "landuse")
    missing = [
        name for name in required_layers
        if osm_layer_metadata.get(name, {}).get("status") != "available"
    ]
    return ("incomplete_osm_proxies" if missing
            else "osm_proxy_features_observed_coverage_unverified"), missing


def reject_osm_planning_layer_read_errors(osm_layer_metadata):
    """Do not plan with empty geometries produced by a failed cache read."""
    errors = [
        f"{name} ({osm_layer_metadata.get(name, {}).get('reason', 'unknown')})"
        for name in ("buildings", "landuse")
        if osm_layer_metadata.get(name, {}).get("status") == "read_error"
    ]
```

源程序鉴别材料 • 开头连续部分 6/30

```
    if errors:
        raise RuntimeError("OSM 规划图层读取失败，拒绝按空图层继续规划: " + ", ".join(errors))


def route_hazard_exposure(geo, hazard_masks, transform, shape):
    """Report raster-cell contact with hazard masks for the selected route."""
    if not geo or len(geo) < 2:
        return {"status": "route_too_short", "route_cell_count": 0, "layers": {}}
    route_mask = rasterize_on([(LineString(geo), {})], transform, shape) > 0
    route_cells = int(np.count_nonzero(route_mask))
    layers = {}
    for name, mask in hazard_masks.items():
        hazard = np.asarray(mask) > 0
        cells = int(np.count_nonzero(route_mask & hazard))
        layers[name] = {
            "route_cell_count": cells,
            "route_cell_fraction": cells / route_cells if route_cells else None,
        }
    return {
        "status": "measured" if route_cells else "route_not_rasterized",
        "route_cell_count": route_cells,
        "cell_fraction_is_approximate": True,
        "layers": layers,
    }


def route_custom_constraint_exposure(geo, constraints, transform, shape):
    """Measure rasterized route contact with configured soft constraint costs."""
    if constraints is None:
        return {"status": "disabled", "route_cell_count": 0,
                "soft_affected_route_cell_count": 0,
                "soft_affected_route_cell_fraction": None,
                "summed_soft_weight_cells": None}
    if not geo or len(geo) < 2:
        return {"status": "route_too_short", "route_cell_count": 0,
                "soft_affected_route_cell_count": 0,
                "soft_affected_route_cell_fraction": None,
                "summed_soft_weight_cells": 0.0}
    soft_cost = np.asarray(constraints["soft_cost"], dtype=np.float64)
    if soft_cost.shape != tuple(shape):
        raise ValueError("custom constraint raster does not match route grid")
    route_mask = rasterize_on([(LineString(geo), {})], transform, shape) > 0
    route_cell_count = int(np.count_nonzero(route_mask))
    touched_soft_cost = soft_cost[route_mask]
    affected_count = int(np.count_nonzero(touched_soft_cost > 0.0))
    return {
        "status": "measured" if route_cell_count else "route_not_rasterized",
        "route_cell_count": route_cell_count,
        "soft_affected_route_cell_count": affected_count,
        "soft_affected_route_cell_fraction": (
```

源程序鉴别材料 • 开头连续部分 7/30

```
            affected_count / route_cell_count if route_cell_count else None),
        "summed_soft_weight_cells": float(touched_soft_cost.sum()),
        "fraction_denominator": "all_touched_route_cells; approximate, not length weighted",
    }


def route_population_coverage(geo, valid_population_mask, transform, shape):
    """Report whether a route crosses cells with unknown population values."""
    if valid_population_mask is None:
        return {"status": "disabled", "route_cell_count": 0,
                "unknown_route_cell_count": 0, "coverage_ratio": None}
    if not geo or len(geo) < 2:
        return {"status": "route_too_short", "route_cell_count": 0,
                "unknown_route_cell_count": 0, "coverage_ratio": None}
    route_mask = rasterize_on([(LineString(geo), {})], transform, shape) > 0
    route_cells = int(np.count_nonzero(route_mask))
    unknown_cells = int(np.count_nonzero(
        route_mask & ~np.asarray(valid_population_mask, dtype=bool)))
    return {
        "status": "complete" if unknown_cells == 0 else "incomplete_unknown_population_cells",
        "route_cell_count": route_cells,
        "unknown_route_cell_count": unknown_cells,
        "coverage_ratio": (1.0 - unknown_cells / route_cells if route_cells else None),
        "cell_fraction_is_approximate": True,
    }


def validate_endpoint_cells(start, end, shape, hard_mask, blocking_layers=None):
    """Require fixed endpoints to lie inside the DEM and the complete passable mask."""
    height, width = shape
    for label, (row, col) in (("起点", start), ("终点", end)):
        if not (0 <= row < height and 0 <= col < width):
            raise ValueError(f"{label}超出 DEM 范围")
        if not hard_mask[row, col]:
            reasons = [name for name, mask in (blocking_layers or {}).items()
                       if mask is not None and bool(mask[row, col])]
            if reasons:
                reason_text = "、".join(reasons)
                raise ValueError(f"{label}位于不可通行像元（{reason_text}）；请核对接入点坐标或约束")
            raise ValueError(f"{label}位于 NoData/建筑/配置禁区内；请核对接入点坐标或约束")
    return start, end


def build_tower_masks(dem, slope, custom_hard_mask=None,
                      max_surface_slope_deg=None, max_terrain_elevation_m=None,
                      building_mask=None, building_distance_m=None,
                      building_clearance_m=0.0):
    """Separate overhead-span traversability from candidate tower-site limits."""
    elevation = np.asarray(dem)
    surface_slope = np.asarray(slope)
```

源程序鉴别材料 • 开头连续部分 8/30

```
    if elevation.ndim != 2 or surface_slope.shape != elevation.shape:
        raise ValueError("dem and slope must be same-shaped 2D arrays")
    span_mask = np.isfinite(elevation)
    for mask, label in ((custom_hard_mask, "custom_hard_mask"),
                        (building_mask, "building_mask")):
        if mask is not None:
            values = np.asarray(mask)
            if values.shape != elevation.shape:
                raise ValueError(f"{label} must match the DEM")
            span_mask &= values == 0
    if max_terrain_elevation_m is not None:
        span_mask &= elevation <= max_terrain_elevation_m
    if building_clearance_m > 0 and building_distance_m is not None:
        distance = np.asarray(building_distance_m)
        if distance.shape != elevation.shape:
            raise ValueError("building_distance_m must match the DEM")
        span_mask &= distance > building_clearance_m

    site_mask = span_mask.copy()
    if max_surface_slope_deg is not None:
        site_mask &= surface_slope <= max_surface_slope_deg
    return span_mask.astype(np.uint8), site_mask.astype(np.uint8)


def _geometry_intersects_features(geometry, features):
    """Check exact intersections against OSM geometry rows or bare geometries."""
    return _geometry_intersection_count(geometry, features) > 0


def _geometry_intersection_summary(geometry, features):
    """Count intersections and repaired nearby OSM geometries."""
    minx, miny, maxx, maxy = geometry.bounds
    count = 0
    repaired = 0
    for feature in features:
        obstacle = feature[0] if isinstance(feature, (tuple, list)) else feature
        left, bottom, right, top = obstacle.bounds
        if left <= maxx and right >= minx and bottom <= maxy and top >= miny:
            if not obstacle.is_valid:
                obstacle = make_valid(obstacle)
                if obstacle.is_empty or not obstacle.is_valid:
                    raise ValueError("相交建筑几何修复失败，拒绝通过精确建筑复核")
                repaired += 1
            if geometry.intersects(obstacle):
                count += 1
    return count, repaired


def _geometry_intersection_count(geometry, features):
    """Count mapped features intersecting a geometry after validity repair."""
```

源程序鉴别材料 • 开头连续部分 9/30

```
    return _geometry_intersection_summary(geometry, features)[0]


def open_safe_building_endpoint_pixels(endpoints, hard_mask, blocking_layers,
                                       building_geometries, transform,
                                       building_clearance_m=0.0):
    """Open only endpoint cells blocked by rasterization when exact geometry is clear."""
    if not building_geometries or building_clearance_m > 0:
        return []

    exceptions = []
    for label, (row, col), point_xy in endpoints:
        if not (0 <= row < hard_mask.shape[0] and 0 <= col < hard_mask.shape[1]):
            continue
        if hard_mask[row, col]:
            continue
        reasons = [name for name, mask in blocking_layers.items()
                   if mask is not None and bool(mask[row, col])]
        if reasons != ["OSM 建筑足迹"]:
            continue

        point = Point(*point_xy)
        if _geometry_intersects_features(point, building_geometries):
            continue
        center_xy = transform * (col + 0.5, row + 0.5)
        connector = LineString([point_xy, center_xy])
        if _geometry_intersects_features(connector, building_geometries):
            continue

        probe_mask = hard_mask.copy()
        probe_mask[row, col] = 1
        if path_hard_violations([point_xy, center_xy], transform, probe_mask):
            continue
        exceptions.append({"label": label, "row": row, "col": col,
                           "point_xy": tuple(point_xy)})

    for exception in exceptions:
        hard_mask[exception["row"], exception["col"]] = 1
    return exceptions


def validate_route_against_buildings(geo, building_geometries):
    """Fail closed when an exported route intersects an available building footprint."""
    if not geo:
        return None, {"building_geometry_status": "no_path",
                      "building_geometry_intersections": 0}
    route = LineString(geo) if len(geo) > 1 else Point(*geo[0])
    intersections, repaired = _geometry_intersection_summary(route, building_geometries)
    report = {
        "building_geometry_status": "fail" if intersections else "pass",
```

源程序鉴别材料 • 开头连续部分 10/30

```
        "building_geometry_intersections": intersections,
        "building_geometry_repaired_features": repaired,
    }
    return (None if intersections else geo), report


def annotate_route_validation_context(report, endpoint_exceptions, building_metadata):
    """Record endpoint exceptions and the scope of the available building check."""
    report["endpoint_rasterization_exceptions"] = [
        {"label": item["label"], "row": int(item["row"]),
         "col": int(item["col"]), "point_xy": list(item["point_xy"])}
        for item in endpoint_exceptions
    ]
    report["building_geometry_coverage_status"] = building_metadata.get(
        "coverage_status", "unknown")
    report["building_geometry_coverage_reason"] = building_metadata.get(
        "coverage_reason")
    report["building_geometry_check_scope"] = (
        "loaded_osm_features_only"
        if building_metadata.get("status") == "available" else "unavailable")
    report["building_geometry_source_repair_count"] = int(
        building_metadata.get("geometry_repaired_count", 0))
    report["building_geometry_skipped_nonfinite_count"] = int(
        building_metadata.get("nonfinite_geometry_outside_bbox_count", 0))
    return report


def run_method_or_skip_on_oom(method_name, run_method):
    """Skip a neural candidate only for PyTorch OOM; let other errors surface."""
    try:
        return run_method(), None
    except torch.OutOfMemoryError as exc:
        message = str(exc)
    print(f"  [{method_name}] 显存不足，跳过该方法并继续其余规划: {message}")
    if cuda_is_usable(torch):
        torch.cuda.empty_cache()
    return None, message


def train_vin_or_skip_on_oom(*args, **kwargs):
    """Skip VIN on PyTorch OOM without suppressing unrelated failures."""
    result, _ = run_method_or_skip_on_oom("VIN-Grad", lambda: train_vin(*args, **kwargs))
    return result


def validate_unique_case_names(cases, check_reference_names=False):
    names = [case[0] for case in cases]
    if len(set(names)) != len(names):
        raise ValueError("用例名称必须唯一，重复名称会覆盖输出目录")
    if check_reference_names:
```

源程序鉴别材料 • 开头连续部分 11/30

```
        # Reference files must resolve uniquely on Windows as well as POSIX.
        reference_names = [safe_reference_name(name).casefold() for name in names]
        if len(set(reference_names)) != len(reference_names):
            raise ValueError("用例名净化后发生重复，无法安全匹配逐任务参考线路文件")


def validate_cases_available(cases):
    if not cases:
        raise ValueError("未找到有效规划任务；请检查输入 CSV 或站点任务")
    if any(tuple(start) == tuple(end) for _, start, end in cases):
        raise ValueError("起终点坐标相同，不能生成零长度线路")


def load_case_reference(path, require_single_line=True):
    if not path or not Path(path).is_file():
        return None, "reference_missing"
    try:
        real_geo = load_real_path(
            path, require_single_line=require_single_line, raise_errors=True)
    except ValueError as exc:
        print(f"  [warn] 目标线路参考文件无效: {exc}")
        return None, "reference_invalid_or_empty"
    except Exception as exc:
        print(f"  [warn] 目标线路参考文件无法读取: {exc}")
        return None, "reference_invalid_or_unreadable"
    return (real_geo, "reference_loaded" if real_geo else "reference_invalid_or_empty")


def summarize_run_completion(expected_cases, results, failed_cases):
    failed_names = {item["case"] for item in failed_cases}
    no_route_cases = [
        record["case"] for record in results
        if record["metrics"].get(FINAL_ROUTE_LABEL, {}).get("planning_status")
        == "no_hard_constraint_compliant_route"
    ]
    no_route_names = set(no_route_cases)
    completed = {
        record["case"] for record in results
        if record["case"] not in failed_names and record["case"] not in no_route_names
    }
    exploratory_cases = [
        record["case"] for record in results
        if record["case"] in completed
        and record["metrics"].get(FINAL_ROUTE_LABEL, {}).get("planning_status")
        == "exploratory_only_unverified_endpoints"
    ]
    return {
        "expected_cases": expected_cases,
        "completed_cases": len(completed),
        "recorded_cases": len(results),
```

源程序鉴别材料 • 开头连续部分 12/30

```
        "failed_cases": failed_cases,
        "no_route_cases": no_route_cases,
        "exploratory_cases": exploratory_cases,
        "complete": expected_cases > 0 and len(completed) == expected_cases
        and not failed_cases and not no_route_cases,
    }


def route_planning_status(final_geo, station_task, missing_screening_layers):
    if not final_geo:
        return "no_hard_constraint_compliant_route"
    if station_task and station_task["endpoint_qualification"] != \
            "declared_verified_line_connection_points":
        return "exploratory_only_unverified_endpoints"
    if missing_screening_layers:
        return "screening_candidate_data_incomplete"
    return "screening_candidate"


def format_case_error(exc):
    message = str(exc).strip()
    return f"{type(exc).__name__}: {message}" if message else type(exc).__name__


def method_visual_similarity_summary(results, method, expected_cases=None, failed_case_names=None):
    excluded = set(failed_case_names or ())
    eligible = [record for record in results
                if record["case"] not in excluded
                and record["metrics"].get(FINAL_ROUTE_LABEL, {}).get("planning_status")
                != "no_hard_constraint_compliant_route"]
    comparisons = [record["metrics"].get(method, {}).get("route_similarity")
                   for record in eligible]
    comparisons = [value for value in comparisons if value]
    total_cases = max(len(results), expected_cases or 0)
    p95 = [value["symmetric"]["p95_m"] for value in comparisons]
    hausdorff = [value["symmetric"]["sampled_hausdorff_m"] for value in comparisons]
    coverage = [value["symmetric"]["coverage_f1_within_m"]["1000"]
                for value in comparisons]
    coverage_100m = [value["symmetric"]["coverage_f1_within_m"].get("100")
                     for value in comparisons]
    coverage_100m = [value for value in coverage_100m if value is not None]
    length_error = [value["length"]["absolute_difference_pct"] for value in comparisons]
    return {
        "measured": len(comparisons),
        "unmeasured": max(0, total_cases - len(comparisons)),
        "median_symmetric_p95_m": float(np.median(p95)) if p95 else None,
        "median_sampled_hausdorff_m": float(np.median(hausdorff)) if hausdorff else None,
        "median_coverage_f1_100m": float(np.median(coverage_100m)) if coverage_100m else None,
        "median_coverage_f1_1km": float(np.median(coverage)) if coverage else None,
        "median_length_difference_pct": float(np.median(length_error)) if length_error else None,
```

源程序鉴别材料 • 开头连续部分 13/30

```
    }


def validate_real_shp_argument(real_shp, cases):
    if real_shp is None:
        return
    if not Path(real_shp).is_file():
        raise ValueError("--real-shp 必须指向一个存在的目标线路文件")
    if len(cases) != 1:
        raise ValueError("--real-shp 只支持单任务目标线路；多任务请使用 --real-dir 提供逐任务线路")


def configure_console_output():
    for stream in (sys.stdout, sys.stderr):
        reconfigure = getattr(stream, "reconfigure", None)
        if reconfigure:
            reconfigure(errors="replace")


def parse_lat_lon(value, label):
    try:
        parts = [float(part) for part in value.split(",")]
    except ValueError as exc:
        raise ValueError(f"{label}必须是纬度,经度数值") from exc
    if (len(parts) != 2 or not all(math.isfinite(part) for part in parts)
            or not -90 <= parts[0] <= 90 or not -180 <= parts[1] <= 180):
        raise ValueError(f"{label}经纬度无效；纬度须在[-90,90]、经度须在[-180,180]")
    return tuple(parts)


def main():
    configure_console_output()
    t0 = time.time()
    ap = argparse.ArgumentParser(description="复杂山区输电线路启发式路径规划 (V5.0)")
    ap.add_argument("--dem", type=str, default=None)
    ap.add_argument("--dem-vertical-datum", type=str, default=None,
                    help="用于净空筛查的 DEM 垂直基准声明；须与站点实测高程基准一致才会使用实测端点高程")
    ap.add_argument("--start", type=str, default=None, help="起点 lat,lon")
    ap.add_argument("--end", type=str, default=None, help="终点 lat,lon")
    ap.add_argument("--stations-csv", type=str, default=None,
                    help="站点表需含 station_id,lat,lon,coordinate_crs；正式任务还需声明核验过的线路接入点")
    ap.add_argument("--route-tasks-csv", type=str, default=None,
                    help="站间任务表: route_id,from_station_id,to_station_id")
    ap.add_argument("--allow-unverified-endpoints", action="store_true",
                    help="允许未核验端点仅作探索性规划；输出不得解释为正式站间候选")
    ap.add_argument("--constraint-geojson", type=str, default=None,
                    help="可选 WGS84 多边形图层; properties 需含 mode=hard/soft、source；soft 还需 weight。选塔时硬约束另需核验声明和凭证")
    ap.add_argument("--land-mask-geojson", type=str, default=None,
                    help="可选 WGS84 陆域 Polygon/MultiPolygon；多边形外区域作为硬不可通行区")
    default_hazard_dir = PACKAGE_DIR / "data" / "official_sources" / "ardswc_115_mvt_z10_20261023"
```

源程序鉴别材料 • 开头连续部分 14/30

```
    ap.add_argument("--official-hazards-dir", type=str, default=str(default_hazard_dir),
                    help="农业部 115 年土石流/大规模崩塌 MVT 数据目录；默认读取软件包内离线图层")
    ap.add_argument("--official-hazard-weight", type=float, default=1.0,
                    help="官方土石流/崩塌图层联合软代价权重；重叠类别不重复累计，0 表示关闭")
    ap.add_argument("--turn-penalty-weight", type=float, default=0.0,
                    help="A* 转弯平滑代价系数；0 关闭。启用后使用 8 方向状态、内存占用显著增加")
    ap.add_argument("--settlement-weight", type=float, default=2.0,
                    help="OSM 建筑密度/住宅与商业用地代理的软避让权重(方案筛选参数，默认 2.0)")
    ap.add_argument("--population-raster", type=str, default=None,
                    help="可选单波段 EPSG:4326 人口数栅格；值须为每像元人口估算数")
    ap.add_argument("--population-weight", type=float, default=0.0,
                    help="WorldPop 人口密度软成本权重；默认 0 关闭，需同时指定 --population-raster")
    ap.add_argument("--population-nodata-policy", choices=("maximum", "median", "zero"),
                    default="maximum",
                    help="人口栅格 NoData 搜索代价填充值；默认 maximum 保守惩罚，median/zero 仅作缺失敏感性分析")
    ap.add_argument("--ridge-penalty-weight", type=float, default=0.10,
                    help="基于 300 米 TPI 的高脊软惩罚权重；仅用地形，不读取线路几何")
    ap.add_argument("--landcover-penalty-weight", type=float, default=0.0,
                    help="OSM 林地/湿地软代价权重；默认 0 关闭，不完整环境图层仅作可选筛查")
    ap.add_argument("--building-clearance-m", type=float, default=0.0,
                    help="建筑物栅格硬避让距离(米，必须小于 5000)；0 仅屏蔽已映射建筑足迹，不代表规范净距")
    ap.add_argument("--grade-penalty-weight", type=float, default=1.0,
                    help="沿线相邻 DEM 高差软惩罚权重(以 30%%地面坡度作为一个参考级)")
    ap.add_argument("--terrain-slope-weight", type=float, default=0.75,
                    help="DEM 坡度面软代价权重；默认 0.75，0 可用于消融测试")
    ap.add_argument("--roughness-weight", type=float, default=0.25,
                    help="900 米地形粗糙度软代价权重；默认 0.25，0 可用于消融测试")
    ap.add_argument("--astar-heuristic-weight", type=float, default=1.0,
                    help="A*启发函数权重；1.0 对当前离散代价面求最优，大于 1 时为近似搜索且不保证误差界")
    ap.add_argument("--experimental-theta-star", action="store_true",
                    help="额外生成 Theta* 实验候选；复用 A* 栅格代价面，不计软坡度/转弯代价，"
                         "经常规约束复核但不参与最终选路")
    ap.add_argument("--experimental-astar-16", action="store_true",
                    help="额外运行同目标的 8/16 邻域精确积分 A* 对照；复用含地形与约束图层的代价面，"
                         "关闭相邻地面坡度附加惩罚和转弯惩罚，"
                         "配置的地面坡度上限仍作硬约束；均不参与最终选路")
    ap.add_argument("--experimental-astar-16-shared-objective", action="store_true",
                    help="额外运行共享边成本公式的 8/16 邻域 A* 对照：精确积分同一代价面并计入端点地面坡度惩罚；"
                         "不含转弯惩罚，配置的地面坡度上限仍作硬约束；均不参与最终选路")
    ap.add_argument("--experimental-sampled-pareto", action="store_true",
                    help="对同一地形/约束代价面采样生成 A* 多目标候选；只输出实验候选，不参与最终选路")
    ap.add_argument("--experimental-tower-graph", action="store_true",
                    help="额外生成带候选塔位和档距约束的实验路线；默认仅作筛选对照")
    ap.add_argument("--select-experimental-tower-graph", action="store_true",
                    help="显式把通过档距、硬约束、建筑及逐跨 DEM 净空筛查的塔位图候选作为最终筛查方案")
    ap.add_argument("--tower-min-span-m", type=float, default=None,
                    help="实验塔位图最小水平档距；必须按具体线路工程参数提供")
    ap.add_argument("--tower-max-span-m", type=float, default=None,
                    help="实验塔位图最大水平档距；必须按具体线路工程参数提供")
    ap.add_argument("--tower-candidate-stride", type=int, default=2,
```

源程序鉴别材料 • 开头连续部分 15/30

```
                    help="塔位候选像元步长；会影响可达性，默认 2，仅为栅格采样参数")
    ap.add_argument("--tower-attachment-height-m", type=float, default=None,
                    help="可选净空筛查参数：固定挂点高度；需与弧垂和最小净空同时提供")
    ap.add_argument("--tower-max-midspan-sag-m", type=float, default=None,
                    help="可选净空筛查参数：固定最大跨中弧垂；不代表导线荷载计算")
    ap.add_argument("--tower-min-clearance-m", type=float, default=None,
                    help="可选净空筛查参数：固定最小地面净空；通过筛查不代表工程认证")
    ap.add_argument("--max-ground-grade", type=float, default=None,
                    help="可选地面相邻栅格最大坡度比，如 0.30；属于筛选阈值，不是线路设计限值")
    ap.add_argument("--max-surface-slope-deg", type=float, default=None,
                    help="可选地表坡度硬限制(度)；默认不设工程阈值")
    ap.add_argument("--max-terrain-elevation-m", type=float, default=None,
                    help="可选地形海拔硬限制(米)；默认不设工程阈值")
    ap.add_argument("--use-costunet", action="store_true",
                    help="把 CostUNet 作为 50%%软先验混入规则代价；默认关闭，避免未知训练来源影响基线")
    ap.add_argument("--max-side", type=int, default=WORK_MAX_SIDE,
                    help=f"规划网格最长边；默认{WORK_MAX_SIDE}，尽量保留 DEM 原始分辨率，内存不足时可调低")
    ap.add_argument("--window-margin-deg", type=float, default=0.30,
                    help="起终点经纬度包围框外扩度数；扩大可检查候选路线是否受搜索窗口边界限制")
    ap.add_argument("--cases", type=int, default=4)
    ap.add_argument("--include-ai-diagnostics", action="store_true",
                    help="可选运行 VIN/PPO 作为诊断对照；AI 候选不会被选为最终路线")
    ap.add_argument("--no-ppp", action="store_true", help="在 --include-ai-diagnostics 下跳过 PPO")
    ap.add_argument("--no-vin", action="store_true", help="在 --include-ai-diagnostics 下跳过 VIN-Grad")
    ap.add_argument("--out-dir", type=str, default=None)
    ap.add_argument("--with-real", action="store_true",
                    help="加载参考线路，仅在规划选路完成后计算 v2 双向空间相似度")
    ap.add_argument("--real-cases", action="store_true",
                    help="改用 v3 的 4 条真实走廊(R01~R04)并启用真实线对照")
    ap.add_argument("--real-cases-csv", type=str, default=None,
                    help="从真实走廊 CSV(列: name,start_lat,start_lon,end_lat,end_lon)载入测试用例；"
                         "默认在 CSV 同目录查找参考线，可用 --real-dir 指定其他目录。真实线只用于规划后的对照。")
    ap.add_argument("--real-dir", type=str, default=None,
                    help="逐任务参考线目录；每个匹配 SHP 必须恰含一个 LineString 资产")
    ap.add_argument("--real-shp", type=str, default=None,
                    help="单任务指定只含一个 LineString 且 CRS 明确的目标线路 SHP 作为事后对照；"
                         "多任务请用 --real-dir 按任务分别提供线路。规划阶段不会读取该线。")
    ap.add_argument("--osm-cache-dir", type=str, default=None,
                    help="OSM 环境图层缓存目录，内含 taiwan_buildings.pkl、taiwan_landuse.pkl 等文件")
    ap.add_argument("--rebuild-osm-cache", action="store_true",
                    help="调用 shared.data_acquisition 重新生成台湾 OSM 缓存 pkl(覆盖错误区域数据)，"
                         "然后继续正常流程。首次使用或土地利用波段恒为 0 时请启用。")
    # PPO 强度(显存/耗时敏感，默认中等；RTX 4060 8GB 可调低防 OOM)
    ap.add_argument("--ppo-updates", type=int, default=60, help="PPO 训练更新轮数(默认 60)")
    ap.add_argument("--ppo-episodes", type=int, default=8, help="PPO 每轮采样的轨迹数(默认 8)")
    ap.add_argument("--ppo-epochs", type=int, default=4, help="PPO 每轮策略优化 epoch(默认 4)")
    ap.add_argument("--ppo-bc", type=int, default=60, help="PPO 行为克隆预热步数(默认 60)")
    args = ap.parse_args()
    if bool(args.stations_csv) != bool(args.route_tasks_csv):
        ap.error("--stations-csv 和 --route-tasks-csv 必须同时提供")
```

源程序鉴别材料 • 开头连续部分 16/30

```
    if args.select_experimental_tower_graph:
        if not args.experimental_tower_graph:
            ap.error("--select-experimental-tower-graph 必须同时启用 --experimental-tower-graph")
        if not args.stations_csv:
            ap.error("选择塔位图方案必须使用站点 CSV 任务")
        if not args.constraint_geojson:
            ap.error("选择塔位图方案必须提供含核验凭证的 --constraint-geojson 硬约束")
        if any(value is None for value in (
                args.tower_attachment_height_m,
                args.tower_max_midspan_sag_m,
                args.tower_min_clearance_m)) or not (
                    args.dem_vertical_datum and args.dem_vertical_datum.strip()):
            ap.error("选择塔位图方案必须提供完整的塔位净空参数和 --dem-vertical-datum")
    if args.allow_unverified_endpoints and not args.stations_csv:
        ap.error("--allow-unverified-endpoints 仅适用于站点 CSV 任务")
    if bool(args.start) != bool(args.end):
        ap.error("--start 和 --end 必须同时提供")
    task_sources = sum((
        bool(args.stations_csv), bool(args.start or args.end),
        bool(args.real_cases_csv), bool(args.real_cases),
    ))
    if task_sources > 1:
        ap.error("任务输入来源只能选一种：站点 CSV、--start/--end、--real-cases-csv 或 --real-cases")
    if args.real_shp and (args.real_cases_csv or args.real_cases):
        ap.error("--real-shp 不能与 --real-cases-csv/--real-cases 同时使用")
    if args.real_shp and args.real_dir:
        ap.error("--real-shp 不能与 --real-dir 同时使用")
    if args.max_side < 1:
        ap.error("--max-side 必须是正整数")
    if args.window_margin_deg <= 0:
        ap.error("--window-margin-deg 必须大于 0")
    float_parameters = {
        "--building-clearance-m": args.building_clearance_m,
        "--ridge-penalty-weight": args.ridge_penalty_weight,
        "--landcover-penalty-weight": args.landcover_penalty_weight,
        "--settlement-weight": args.settlement_weight,
        "--population-weight": args.population_weight,
        "--grade-penalty-weight": args.grade_penalty_weight,
        "--terrain-slope-weight": args.terrain_slope_weight,
        "--roughness-weight": args.roughness_weight,
        "--official-hazard-weight": args.official_hazard_weight,
        "--turn-penalty-weight": args.turn_penalty_weight,
        "--astar-heuristic-weight": args.astar_heuristic_weight,
        "--window-margin-deg": args.window_margin_deg,
        "--max-ground-grade": args.max_ground_grade,
        "--max-surface-slope-deg": args.max_surface_slope_deg,
        "--max-terrain-elevation-m": args.max_terrain_elevation_m,
        "--tower-min-span-m": args.tower_min_span_m,
        "--tower-max-span-m": args.tower_max_span_m,
        "--tower-attachment-height-m": args.tower_attachment_height_m,
```

源程序鉴别材料 • 开头连续部分 17/30

```
        "--tower-max-midspan-sag-m": args.tower_max_midspan_sag_m,
        "--tower-min-clearance-m": args.tower_min_clearance_m,
    }
    for option, value in float_parameters.items():
        if value is not None and not math.isfinite(value):
            ap.error(f"{option} 必须是有限数值")
    if (args.building_clearance_m < 0 or args.ridge_penalty_weight < 0
            or args.landcover_penalty_weight < 0
            or args.settlement_weight < 0
            or args.population_weight < 0
            or args.grade_penalty_weight < 0
            or args.terrain_slope_weight < 0
            or args.roughness_weight < 0
            or args.official_hazard_weight < 0
            or args.turn_penalty_weight < 0):
        ap.error("避让距离与代价权重不能为负数")
    if args.astar_heuristic_weight < 1:
        ap.error("--astar-heuristic-weight 必须大于或等于 1.0")
    if (args.experimental_astar_16_shared_objective
            and args.turn_penalty_weight > 0):
        ap.error("--experimental-astar-16-shared-objective 暂不支持非零 --turn-penalty-weight")
    if args.population_weight > 0 and not args.population_raster:
        ap.error("启用 --population-weight 时必须同时指定 --population-raster")
    if args.population_raster and not Path(args.population_raster).is_file():
        ap.error(f"--population-raster 文件不存在: {args.population_raster}")
    if args.population_weight > 0 and not HAS_RASTERIO:
        ap.error("启用人口软成本需要 rasterio")
    if args.building_clearance_m >= 5000:
        ap.error("--building-clearance-m 必须小于 5000 米（当前距离栅格上限）")
    if args.max_ground_grade is not None and args.max_ground_grade <= 0:
        ap.error("--max-ground-grade 必须大于 0")
    tower_profile = (args.tower_attachment_height_m,
                     args.tower_max_midspan_sag_m,
                     args.tower_min_clearance_m)
    if args.experimental_tower_graph:
        if args.tower_min_span_m is None or args.tower_max_span_m is None:
            ap.error("--experimental-tower-graph 必须显式提供 --tower-min-span-m 和 --tower-max-span-m")
        if (args.tower_min_span_m < 0
                or args.tower_max_span_m <= args.tower_min_span_m):
            ap.error("塔位图档距必须满足 0 <= 最小档距 < 最大档距")
        if args.tower_candidate_stride < 1:
            ap.error("--tower-candidate-stride 必须是正整数")
        if any(value is not None for value in tower_profile) and any(
                value is None for value in tower_profile):
            ap.error("净空筛查必须同时提供挂点高度、最大跨中弧垂和最小净空")
    elif (args.tower_min_span_m is not None or args.tower_max_span_m is not None
          or any(value is not None for value in tower_profile)):
        ap.error("塔位图参数必须与 --experimental-tower-graph 同时使用")

    if args.rebuild_osm_cache:
```

源程序鉴别材料 • 开头连续部分 18/30

```
        print("[OSM] 请求重建台湾 OSM 缓存 ...")
        try:
            from . import rebuild_osm_cache as _roc
            _roc.rebuild()
        except Exception as _e:
            print(f"[warn] OSM 缓存重建失败: {_e}")
        print("[OSM] 缓存重建完成, 继续后续流程。")

    OUT = Path(args.out_dir) if args.out_dir else OUTPUTS_DIR
    rng = np.random.default_rng(RANDOM_SEED)

    print("=" * 70)
    print("  复杂山区输电线路启发式路径规划 V5.0")
    print("=" * 70)

    dem_path = Path(args.dem) if args.dem else find_dem()
    if dem_path is None:
        sys.exit("[FATAL] 未找到 DEM, 请用 --dem 指定。")
    print(f"[DEM] {dem_path}")

    station_task_by_route = {}
    if args.stations_csv:
        try:
            station_tasks = load_station_tasks(
                args.stations_csv, args.route_tasks_csv,
                allow_unverified=args.allow_unverified_endpoints)
        except (OSError, ValueError) as exc:
            ap.error(str(exc))
        if (args.select_experimental_tower_graph
                and any(task["endpoint_qualification"]
                        != "declared_verified_line_connection_points"
                        for task in station_tasks)):
            ap.error("选择塔位图方案要求所有端点声明为已核验线路接入点")
        cases = []
        for task in station_tasks:
            cases.append((task["route_id"], task["start"], task["end"]))
            station_task_by_route[task["route_id"]] = task
    elif args.start and args.end:
        try:
            start = parse_lat_lon(args.start, "--start")
            end = parse_lat_lon(args.end, "--end")
        except ValueError as exc:
            ap.error(str(exc))
        cases = [("Custom", start, end)]
    else:
        if args.real_cases_csv:
            if not Path(args.real_cases_csv).is_file():
                ap.error(f"--real-cases-csv 文件不存在: {args.real_cases_csv}")
            try:
                csv_cases = load_real_cases_csv(args.real_cases_csv)
```

源程序鉴别材料 • 开头连续部分 19/30

```
            except (OSError, ValueError) as exc:
                ap.error(str(exc))
            # 默认从 CSV 同目录找参考线；显式 --real-dir 时使用指定目录。
            args.real_dir = args.real_dir or str(Path(args.real_cases_csv).parent)
            cases = [(n, s, e) for (n, s, e, _) in csv_cases]
        elif args.real_cases:
            cases = [(n, s, e) for (n, s, e, _) in REAL_CASES]
        else:
            cases = DEFAULT_CASES[:max(1, args.cases)]

    allow_global_reference_mapping = not any((
        args.real_dir, args.stations_csv, args.start, args.real_cases_csv))
    args.real_dir = args.real_dir or str(REAL_PATH_DIR)
    try:
        validate_cases_available(cases)
        validate_unique_case_names(
            cases,
            check_reference_names=not args.real_shp and bool(
                args.real_cases_csv or args.real_cases or args.with_real),
        )
        validate_real_shp_argument(args.real_shp, cases)
    except ValueError as exc:
        ap.error(str(exc))

    global_real_exists = args.real_shp is not None
    use_real = bool(args.with_real or args.real_cases or args.real_cases_csv or global_real_exists)
    if use_real:
        mode = "单任务目标线路" if global_real_exists else args.real_dir
        print(f"[真实对比] 已启用: 模式={mode}；真实线仅用于规划后的事后评估")

    run_provenance = capture_run_provenance(
        Path(__file__).resolve().parents[1], args, cases, dem_path,
        DOWNLOADED_DIR, model_weights=MODEL_WEIGHTS, has_torch=HAS_TORCH,
        effective_runtime={
            "random_seed": RANDOM_SEED,
            "has_torch": HAS_TORCH,
            "has_scipy": HAS_SCIPY,
            "has_rasterio": HAS_RASTERIO,
            "has_geopandas": HAS_GPD,
        })
    OUT.mkdir(parents=True, exist_ok=True)

    # CostUNet（全局加载一次）
    costunet = load_costunet() if args.use_costunet and HAS_TORCH else None
    if args.use_costunet and costunet is None:
        print("[warn] CostUNet 不可用，继续使用规则代价面")

    all_results = []
    main_done = False
```

源程序鉴别材料 • 开头连续部分 20/30

```
    failed_cases = []
    for ci, (cname, (slat, slon), (elat, elon)) in enumerate(cases):
        print(f"\n--- 用例 {ci + 1}/{len(cases)}: {cname} ---")
        try:
            margin = args.window_margin_deg
            minlon, maxlon = min(slon, elon) - margin, max(slon, elon) + margin
            minlat, maxlat = min(slat, elat) - margin, max(slat, elat) + margin
            dem, transform, crs, pm = load_dem_window(dem_path, minlon, minlat, maxlon, maxlat, args.max_side)
            H, W = dem.shape
            population_proxy = None
            population_valid_mask = None
            population_metadata = {
                "status": "disabled_weight_zero",
                "source_path": (str(Path(args.population_raster).resolve())
                                if args.population_raster else None),
                "weight": args.population_weight,
                "normalization_scale_people_per_km2": (
                    POPULATION_DENSITY_NORMALIZATION_PEOPLE_PER_KM2),
            }
            if args.population_weight > 0:
                dem_valid_mask = np.isfinite(dem)
                population_density, population_metadata = load_population_density_window(
                    args.population_raster, dem.shape, transform, crs,
                    required_mask=dem_valid_mask,
                )
                population_proxy, population_unknown_mask = population_cost_for_routing(
                    population_density, nodata_policy=args.population_nodata_policy,
                    valid_mask=dem_valid_mask)
                population_valid_mask = ~population_unknown_mask
                population_metadata["weight"] = args.population_weight
                population_metadata["source_value_semantics"] = (
                    "modelled people per source cell; not point locations")
                population_metadata["cost_semantics"] = (
                    "log1p(population_density_people_per_km2) / "
                    "log1p(normalization_scale), clipped to [0, 1]")
                population_metadata["nodata_cost_policy"] = {
                    "maximum": "fixed maximum normalized proxy penalty (1.0); not a population estimate",
                    "median": "median normalized proxy among valid population cells on valid DEM pixels in this planning window; imputation only",
                    "zero": "zero-cost fill sensitivity scenario; not an empty-population estimate",
                }[args.population_nodata_policy]
                nodata_policy_label = {
                    "maximum": "最大归一化惩罚",
                    "median": "DEM 有效区域中位数填补（敏感性分析）",
                    "zero": "零代价填补（敏感性分析）",
                }[args.population_nodata_policy]
                print(
                    "  人口软成本已对齐："
                    f"coverage={population_metadata['coverage_ratio']:.3f}, "
                    f"scale={population_metadata['normalization_scale_people_per_km2']:.0f} people/km², "
                    f"NoData 策略={nodata_policy_label}"
```

源程序鉴别材料 • 开头连续部分 21/30

```
        )
    if args.turn_penalty_weight > 0:
        state_gib = H * W * (8 * 8 + 8) / (1024 ** 3)
        if state_gib >= 0.5:
            print(f"  [warn] 转弯状态数组约需 {state_gib:.2f} GiB；还未包含其他栅格和搜索队列")

    sr, sc = geo_to_rowcol(slat, slon, transform)
    er, ec = geo_to_rowcol(elat, elon, transform)
    station_task = station_task_by_route.get(cname)
    shortcut_endpoints = (
        (float(station_task["from_station"]["lon"]),
         float(station_task["from_station"]["lat"])),
        (float(station_task["to_station"]["lon"]),
         float(station_task["to_station"]["lat"])),
    ) if station_task else ((float(slon), float(slat)), (float(elon), float(elat)))

    left = transform.c; top = transform.f
    right = left + transform.a * W; bottom = top + transform.e * H
    bbox = (left, bottom, right, top)
    needs_feature_stack = costunet is not None or (
        args.include_ai_diagnostics and HAS_TORCH and not args.no_vin)
    osm = compute_osm_bands(
        dem, transform, bbox, cache_dir=args.osm_cache_dir,
        include_model_features=needs_feature_stack,
        include_building_distance=args.building_clearance_m > 0,
    )
    osm_layer_metadata = getattr(osm, "metadata", {}).get("layers", {})
    reject_osm_planning_layer_read_errors(osm_layer_metadata)
    terrain = compute_terrain_factors(dem, pm, transform, crs)
    official_hazard_masks, official_hazard_metadata = load_official_hazard_masks(
        args.official_hazards_dir, bbox, transform, dem.shape)
    official_hazard_metadata["soft_weight_applied_in_run"] = args.official_hazard_weight
    if official_hazard_metadata.get("coverage_status") != "complete":
        print("  [warn] 官方灾害图层覆盖不完整；未知瓦片不按无风险处理。"
              f" status={official_hazard_metadata.get('status')}, "
              f"coverage={official_hazard_metadata.get('coverage_status')}")
    custom_constraints = (
        load_user_constraint_overlays(
            args.constraint_geojson, transform, dem.shape,
            require_verified_hard_constraints=args.select_experimental_tower_graph)
        if args.constraint_geojson else None
    )
    custom_constraint_metadata = custom_constraints["metadata"] if custom_constraints else None
    land_mask_layer = (load_land_mask_geojson(
        args.land_mask_geojson, transform, dem.shape)
        if args.land_mask_geojson else None)
    land_mask_metadata = land_mask_layer["metadata"] if land_mask_layer else None
    for layer_name in ("buildings", "landuse"):
        layer = osm_layer_metadata.get(layer_name, {})
        if layer.get("status") != "available":
```

源程序鉴别材料 • 开头连续部分 22/30

```
                print(f"  [warn] OSM {layer_name} 图层状态={layer.get('status', 'unknown')}: "
                      "不能据此认定该范围内没有居民点/建筑")
        stack = build_feature_stack(dem, terrain, osm) if needs_feature_stack else None

        model_cost = costunet.predict(stack) if costunet is not None else None
        cost = build_route_cost(
            terrain, osm, args.settlement_weight, model_cost,
            ridge_penalty_weight=args.ridge_penalty_weight,
            landcover_penalty_weight=args.landcover_penalty_weight,
            official_hazard_masks=official_hazard_masks,
            official_hazard_weight=args.official_hazard_weight,
            terrain_slope_weight=args.terrain_slope_weight,
            roughness_weight=args.roughness_weight,
            population_proxy=population_proxy,
            population_weight=args.population_weight,
        )

        real_geo = None
        if custom_constraints:
            soft_cost = custom_constraints["soft_cost"]
            cost = (cost + soft_cost).astype(np.float32)
        print(f"  代价面范围 [{np.nanmin(cost):.3f}, {np.nanmax(cost):.3f}]")

        slope = terrain["slope"]; elev = terrain["elevation"]
        building_mask = osm.get("building_mask")
        combined_hard_mask = (
            np.asarray(custom_constraints["hard_mask"], dtype=np.uint8).copy()
            if custom_constraints else np.zeros(dem.shape, dtype=np.uint8))
        if land_mask_layer:
            combined_hard_mask = np.maximum(
                combined_hard_mask,
                (land_mask_layer["land_mask"] == 0).astype(np.uint8))
        if args.max_surface_slope_deg is not None:
            if not 0 < args.max_surface_slope_deg <= 90:
                raise ValueError("--max-surface-slope-deg 必须在(0,90]内")
        tower_span_mask, hard_mask = build_tower_masks(
            dem, slope,
            custom_hard_mask=combined_hard_mask,
            max_surface_slope_deg=args.max_surface_slope_deg,
            max_terrain_elevation_m=args.max_terrain_elevation_m,
            building_mask=building_mask,
            building_distance_m=osm.get("building_distance_m"),
            building_clearance_m=args.building_clearance_m,
        )
        endpoint_blockers = {"DEM NoData": ~np.isfinite(dem)}
        if custom_constraints:
            endpoint_blockers["用户硬约束"] = custom_constraints["hard_mask"] > 0
        if land_mask_layer:
            endpoint_blockers["陆域范围外"] = land_mask_layer["land_mask"] == 0
        if args.max_surface_slope_deg is not None:
```

源程序鉴别材料 • 开头连续部分 23/30

```
            endpoint_blockers["地表坡度阈值"] = slope > args.max_surface_slope_deg
        if args.max_terrain_elevation_m is not None:
            endpoint_blockers["地形海拔阈值"] = elev > args.max_terrain_elevation_m
        if building_mask is not None:
            endpoint_blockers["OSM 建筑足迹"] = np.asarray(building_mask) > 0
        if args.building_clearance_m > 0 and "building_distance_m" in osm:
            endpoint_blockers["建筑净距阈值"] = (
                osm["building_distance_m"] <= args.building_clearance_m)
        building_geometries = getattr(osm, "geometries", {}).get("buildings", [])
        endpoint_building_exceptions = open_safe_building_endpoint_pixels(
            (("起点", (sr, sc), (slon, slat)),
             ("终点", (er, ec), (elon, elat))),
            hard_mask, endpoint_blockers, building_geometries, transform,
            args.building_clearance_m,
        )
        for exception in endpoint_building_exceptions:
            tower_span_mask[exception["row"], exception["col"]] = 1
        endpoint_mask = (tower_span_mask if args.experimental_tower_graph
                         else hard_mask)
        validate_endpoint_cells((sr, sc), (er, ec), dem.shape, endpoint_mask,
                                endpoint_blockers)
        same_grid_cell = (sr, sc) == (er, ec)
        cost_grid = (1.0 + 4.0 * cost).astype(np.float64)
        cost_grid = np.where(hard_mask == 0, np.inf, cost_grid)
        tower_span_cost_grid = (1.0 + 4.0 * cost).astype(np.float64)
        tower_span_cost_grid = np.where(
            tower_span_mask > 0, tower_span_cost_grid, np.inf)
        if args.include_ai_diagnostics:
            cost_finite = np.where(np.isfinite(cost), cost, 1.0).astype(np.float32)
            cost_vin = np.where(hard_mask > 0, cost_grid, 0.0).astype(np.float32)
        lat_center = transform.f + transform.e * (H / 2.0)
        route_pixel_spacing = pixel_spacing_meters(transform, lat_center, crs)

        methods = {}
        grade_reports = {}
        V_main = None
        t_vin = 0.0
        t_ppo = 0.0
        t_theta = 0.0
        t_tower_graph = 0.0
        # AI methods are opt-in diagnostics; route selection below excludes them.
        if args.include_ai_diagnostics and HAS_TORCH and not args.no_vin and not same_grid_cell:
            t_v = time.time()
            extra = build_vin_extra(stack)
            goal_mask = np.zeros((H, W), np.float32); goal_mask[er, ec] = 1.0
            torch.manual_seed(RANDOM_SEED)
            V = train_vin_or_skip_on_oom(cost_vin, goal_mask, extra, hard_mask,
                                         route_pixel_spacing, n_steps=80)
            if V is None:
                print("  [VIN-Grad] 本用例未生成 AI 候选")
```

源程序鉴别材料 • 开头连续部分 24/30

```
            else:
                V_main = np.nan_to_num(V, nan=1e6, posinf=1e6, neginf=1e6)
                geo_vin = smooth_path_with_constraints(gradient_track_path(
                    V, cost_finite, hard_mask, (sr, sc), (er, ec), transform),
                    transform, hard_mask, cost_grid, dem,
                    0.0, pixel_spacing=route_pixel_spacing,
                    max_ground_grade=args.max_ground_grade, crs=crs,
                    shortcut_endpoints=shortcut_endpoints)
                geo_vin, grade_reports["VIN-Grad"] = filter_path_by_ground_grade(
                    geo_vin, dem, transform, args.max_ground_grade, crs)
                if geo_vin:
                    grade_reports["VIN-Grad"] = ground_grade_limit_report(
                        geo_vin, dem, transform, args.max_ground_grade, crs)
                else:
                    grade_reports["VIN-Grad"] = ground_grade_limit_report(
                        None, dem, transform, args.max_ground_grade, crs)
                # 保存真实 V/cost/hard/起终点, 便于 PPO 推理调试(无需重跑训练即可秒级重测)
                try:
                    np.savez(str(OUT / "_last_V.npz"), V=V_main, cost=cost_finite,
                             hard=hard_mask, start=np.array([sr, sc]), goal=np.array([er, ec]))
                except Exception:
                    pass
                methods["VIN-Grad"] = geo_vin
                t_vin = time.time() - t_v
                print(f"  [VIN-Grad] 路径 {len(geo_vin) if geo_vin else 0} 点, 耗时 {t_vin:.1f}s")
        else:
            reason = ("未启用 AI 诊断" if not args.include_ai_diagnostics else
                      "起终点位于同一栅格" if same_grid_cell else
                      "已禁用或 torch 不可用")
            print(f"  [skip] VIN-Grad {reason}")

        # --- 方法 2: RL/PPO 策略梯度路径生成器 (每用例独立训练: 代价图与目标不同) ---
        if args.include_ai_diagnostics and HAS_TORCH and not args.no_ppp and not same_grid_cell:
            def build_ppo_candidate():
                torch.manual_seed(RANDOM_SEED + 100)
                print("  [PPO] 训练策略网络 ...")
                t_p = time.time()
                ppo_policy, _ = train_ppo(
                    cost_finite, hard_mask, (er, ec), (sr, sc), value_map=V_main,
                    n_updates=args.ppo_updates, episodes=args.ppo_episodes,
                    ppo_epochs=args.ppo_epochs, bc_steps=args.ppo_bc)
                print(f"  [PPO] 训练完成 {time.time() - t_p:.1f}s")
                ctx = build_ppo_context(np.clip(cost_finite, 0, 1), (er, ec), H, W, value_map=V_main)
                ppo_path = ppo_rollout_path(ppo_policy, ctx, cost_finite, hard_mask, (er, ec), (sr, sc),
                                            n_tries=6, guide_v=V_main, guide_w=6.0, v_tol=0.35)
                geo_ppo = smooth_path_with_constraints(
                    [rowcol_to_geo(r, c, transform) for r, c in ppo_path],
                    transform, hard_mask, cost_grid, dem,
                    0.0, pixel_spacing=route_pixel_spacing,
                    max_ground_grade=args.max_ground_grade, crs=crs,
```

源程序鉴别材料 • 开头连续部分 25/30

```
                    shortcut_endpoints=shortcut_endpoints)
                geo_ppo, grade_reports["PPO"] = filter_path_by_ground_grade(
                    geo_ppo, dem, transform, args.max_ground_grade, crs)
                if geo_ppo:
                    grade_reports["PPO"] = ground_grade_limit_report(
                        geo_ppo, dem, transform, args.max_ground_grade, crs)
                else:
                    grade_reports["PPO"] = ground_grade_limit_report(
                        None, dem, transform, args.max_ground_grade, crs)
                nonlocal t_ppo
                t_ppo = time.time() - t_p
                return geo_ppo

            geo_ppo, ppo_oom = run_method_or_skip_on_oom("PPO", build_ppo_candidate)
            if ppo_oom is None:
                methods["PPO"] = geo_ppo
                print(f"  [PPO] 路径 {len(geo_ppo) if geo_ppo else 0} 点")
        else:
            reason = ("未启用 AI 诊断" if not args.include_ai_diagnostics else
                      "起终点位于同一栅格" if same_grid_cell else
                      "已禁用或 torch 不可用")
            print(f"  [skip] PPO {reason}")

        # Weight 1 is optimal A*. Larger weights are approximate; reopened
        # states do not provide a general optimality bound.
        astar_method = (
            "A*(turn-aware)" if args.turn_penalty_weight > 0 else
            "A*(baseline)" if args.astar_heuristic_weight == 1.0 else
            "A*(weighted)"
        )
        t_astar = time.time()
        a_path = astar_search(
            cost_grid, (sr, sc), (er, ec), elevation=dem, pixel_m=pm,
            grade_penalty_weight=args.grade_penalty_weight,
            max_ground_grade=args.max_ground_grade,
            pixel_spacing=route_pixel_spacing,
            heuristic_weight=args.astar_heuristic_weight,
            turn_penalty_weight=args.turn_penalty_weight,
        )
        raw_astar_geo = ([rowcol_to_geo(r, c, transform) for r, c in a_path]
                         if a_path else None)
        geo_astar = [rowcol_to_geo(r, c, transform) for r, c in a_path] if a_path else None
        if geo_astar:
            geo_astar = smooth_path_with_constraints(
                geo_astar, transform, hard_mask, cost_grid, dem,
                args.grade_penalty_weight, args.turn_penalty_weight,
                route_pixel_spacing, max_ground_grade=args.max_ground_grade,
                crs=crs, shortcut_endpoints=shortcut_endpoints)
        geo_astar, grade_reports[astar_method] = filter_path_by_ground_grade(
            geo_astar, dem, transform, args.max_ground_grade, crs)
```

源程序鉴别材料 • 开头连续部分 26/30

```
            methods[astar_method] = geo_astar
            t_astar = time.time() - t_astar
            print(f"  [{astar_method}] 路径 {len(geo_astar) if geo_astar else 0} 点, "
                  f"路径生成总耗时 {t_astar:.1f}s")

            sampled_pareto_by_method = {}
            if args.experimental_sampled_pareto:
                started = time.time()
                sampled_candidates = sampled_pareto_astar_routes(
                    cost_grid, hard_mask > 0, (sr, sc), (er, ec),
                    pixel_spacing=route_pixel_spacing, elevation=dem, pixel_m=pm,
                    grade_penalty_weight=args.grade_penalty_weight,
                    max_ground_grade=args.max_ground_grade,
                    turn_penalty_weight=args.turn_penalty_weight,
                )
                sample_elapsed = time.time() - started
                for index, candidate in enumerate(sampled_candidates, start=1):
                    method = f"Sampled Pareto {index:02d}"
                    geo = [rowcol_to_geo(row, col, transform)
                           for row, col in candidate["path"]]
                    if geo:
                        geo = smooth_path_with_constraints(
                            geo, transform, hard_mask, cost_grid, dem,
                            args.grade_penalty_weight, args.turn_penalty_weight,
                            route_pixel_spacing,
                            max_ground_grade=args.max_ground_grade, crs=crs,
                            shortcut_endpoints=shortcut_endpoints,
                        )
                    geo, grade_reports[method] = filter_path_by_ground_grade(
                        geo, dem, transform, args.max_ground_grade, crs)
                    methods[method] = geo
                    sampled_pareto_by_method[method] = {
                        "sampled_environmental_weights": candidate[
                            "sampled_environmental_weights"],
                        "objective_vector_on_search_grid": candidate["objective_vector"],
                        "pareto_scope": candidate["pareto_scope"],
                        "search_hard_constraint_legal": candidate[
                            "hard_constraint_legal"],
                        "search_hard_violation_count": candidate[
                            "hard_violation_count"],
                        "candidate_search_batch_time_s": sample_elapsed,
                    }
                    print(f"  [{method}] 网格候选 {len(candidate['path'])} 点；"
                          "之后仍需端点、硬约束和建筑复核")

            experimental_neighbor_results = {}
            neighbor_experiments = []
            if args.experimental_astar_16:
                neighbor_experiments.extend((
                    (ASTAR_8_EXACT_METHOD, NEIGHBORS_8, 0.0),
```

源程序鉴别材料 • 开头连续部分 27/30

```
                    (ASTAR_16_METHOD, NEIGHBORS_16, 0.0),
                ))
            if args.experimental_astar_16_shared_objective:
                neighbor_experiments.extend((
                    (ASTAR_8_SHARED_METHOD, NEIGHBORS_8,
                     args.grade_penalty_weight),
                    (ASTAR_16_SHARED_METHOD, NEIGHBORS_16,
                     args.grade_penalty_weight),
                ))
            for method, neighbors, grade_weight in neighbor_experiments:
                started = time.time()
                raw_path = astar_exact_integral_path(
                    cost_grid, hard_mask, (sr, sc), (er, ec), neighbors,
                    pixel_spacing=route_pixel_spacing, elevation=dem,
                    max_ground_grade=args.max_ground_grade,
                    grade_penalty_weight=grade_weight,
                )
                geo_path = ([rowcol_to_geo(r, c, transform) for r, c in raw_path]
                            if raw_path else None)
                if geo_path:
                    geo_path = smooth_path_with_constraints(
                        geo_path, transform, hard_mask, cost_grid, dem,
                        grade_weight, 0.0, route_pixel_spacing,
                        max_ground_grade=args.max_ground_grade, crs=crs,
                        shortcut_endpoints=shortcut_endpoints,
                    )
                geo_path, grade_reports[method] = filter_path_by_ground_grade(
                    geo_path, dem, transform, args.max_ground_grade, crs)
                elapsed = time.time() - started
                search_raster_cost = integrate_path_cost(
                    raw_path, cost_grid, hard_mask,
                    pixel_spacing=route_pixel_spacing,
                    neighbor_offsets=neighbors,
                ) if raw_path else math.inf
                search_cost = (integrate_path_cost(
                    raw_path, cost_grid, hard_mask,
                    pixel_spacing=route_pixel_spacing,
                    neighbor_offsets=neighbors, elevation=dem,
                    grade_penalty_weight=grade_weight,
                ) if raw_path else math.inf)
                search_grade_penalty_cost = (
                    search_cost - search_raster_cost
                    if math.isfinite(search_cost) and math.isfinite(search_raster_cost)
                    else math.inf)
                methods[method] = geo_path
                experimental_neighbor_results[method] = {
                    "path": raw_path,
                    "neighbors": neighbors,
                    "search_cost": search_cost,
                    "search_raster_cost": search_raster_cost,
```

源程序鉴别材料 • 开头连续部分 28/30

```
                "search_grade_penalty_cost": search_grade_penalty_cost,
                "grade_penalty_weight": grade_weight,
                "objective_scope": (
                    "piecewise_constant_raster_cost_plus_endpoint_grade"
                    if grade_weight > 0.0 else "piecewise_constant_raster_cost_only"),
                "time_s": elapsed,
            }
            print(f"  [{method}] 路径 {len(geo_path) if geo_path else 0} 点, "
                  f"路径生成总耗时 {elapsed:.1f}s；精确积分，"
                  f"软地面坡度权重={grade_weight:g}，转弯权重=0")

        if args.experimental_theta_star:
            t_theta = time.time()
            theta_path = theta_star_cost_path(
                cost_grid, hard_mask, (sr, sc), (er, ec),
                pixel_spacing=route_pixel_spacing, elevation=dem,
                max_ground_grade=args.max_ground_grade)
            geo_theta = ([rowcol_to_geo(r, c, transform) for r, c in theta_path]
                         if theta_path else None)
            if geo_theta:
                geo_theta = smooth_path_with_constraints(
                    geo_theta, transform, hard_mask, cost_grid, dem,
                    0.0, 0.0, route_pixel_spacing,
                    max_ground_grade=args.max_ground_grade, crs=crs,
                    shortcut_endpoints=shortcut_endpoints)
            grade_reports["Theta*(experimental)"] = ground_grade_limit_report(
                geo_theta, dem, transform, args.max_ground_grade, crs)
            methods["Theta*(experimental)"] = geo_theta
            t_theta = time.time() - t_theta
            print(f"  [Theta*(experimental)] 路径 {len(geo_theta) if geo_theta else 0} 点, "
                  f"耗时 {t_theta:.1f}s；仅作对照候选")

        experimental_tower_graph_results = None
        if args.experimental_tower_graph:
            started = time.time()
            tower_profile_args = {}
            clearance_profile_status = "not_requested"
            clearance_profile_ready = True
            if all(value is not None for value in tower_profile):
                endpoint_elevation_overrides = None
                if station_task:
                    endpoint_elevation_overrides, elevation_status = (
                        station_clearance_elevation_overrides(
                            station_task, args.dem_vertical_datum))
                else:
                    elevation_status = "dem_only"
                if elevation_status in (
                        "dem_only",
                        "station_and_dem_vertical_datums_declared_matching"):
                    tower_profile_args = {
```

源程序鉴别材料 • 开头连续部分 29/30

```
                "dem_raster": dem,
                "attachment_height_m": args.tower_attachment_height_m,
                "max_midspan_sag_m": args.tower_max_midspan_sag_m,
                "minimum_clearance_m": args.tower_min_clearance_m,
                **(endpoint_elevation_overrides or {}),
            }
            clearance_profile_status = (
                "dem_only_profile_screening" if elevation_status == "dem_only"
                else "station_elevation_anchored_profile_screening")
        else:
            clearance_profile_status = f"unknown_{elevation_status}"
            clearance_profile_ready = False
    start_col_corner, start_row_corner = ~transform * (slon, slat)
    goal_col_corner, goal_row_corner = ~transform * (elon, elat)
    start_tower_position_rc = (start_row_corner - 0.5,
                               start_col_corner - 0.5)
    goal_tower_position_rc = (goal_row_corner - 0.5,
                              goal_col_corner - 0.5)
    tower_path = None
    if clearance_profile_ready:
        tower_path = tower_graph_route(
            tower_span_cost_grid, tower_span_mask.astype(bool), hard_mask.astype(bool),
            (sr, sc), (er, ec), args.tower_min_span_m,
            args.tower_max_span_m, pixel_spacing=route_pixel_spacing,
            candidate_stride=args.tower_candidate_stride,
            turn_penalty_weight=args.turn_penalty_weight,
            start_position_rc=start_tower_position_rc,
            goal_position_rc=goal_tower_position_rc,
            **tower_profile_args,
        )
    tower_geo = ([rowcol_to_geo(row, col, transform)
                  for row, col in tower_path] if tower_path else None)
    span_clearance_profiles = []
    if tower_path and tower_profile_args:
        for span_index, (first, second) in enumerate(
                zip(tower_path, tower_path[1:])):
            overrides = {}
            for terminal, elevation_key in (
                    (start_tower_position_rc, "start_ground_elevation_m"),
                    (goal_tower_position_rc, "goal_ground_elevation_m")):
                elevation = tower_profile_args.get(elevation_key)
                if elevation is not None:
                    if first == terminal:
                        overrides["start_ground_elevation_m"] = elevation
                    elif second == terminal:
                        overrides["end_ground_elevation_m"] = elevation
            span_clearance_profiles.append({
                "span_index": span_index,
                "start_rc": list(first),
                "end_rc": list(second),
```

源程序鉴别材料 • 开头连续部分 30/30

```
                        **span_clearance_status(
                            first, second, dem, route_pixel_spacing,
                            args.tower_attachment_height_m,
                            args.tower_max_midspan_sag_m,
                            args.tower_min_clearance_m,
                            **overrides),
                    })
        grade_reports[TOWER_GRAPH_METHOD] = overhead_span_grade_report()
        methods[TOWER_GRAPH_METHOD] = tower_geo
        t_tower_graph = time.time() - started
        experimental_tower_graph_results = {
            "path": tower_path,
            "time_s": t_tower_graph,
            "span_min_m": args.tower_min_span_m,
            "span_max_m": args.tower_max_span_m,
            "candidate_stride": args.tower_candidate_stride,
            "turn_penalty_weight": args.turn_penalty_weight,
            "tower_site_max_surface_slope_deg": args.max_surface_slope_deg,
            "clearance_profile_screening": bool(tower_profile_args),
            "clearance_profile_status": clearance_profile_status,
            "clearance_profile_spans": span_clearance_profiles,
            "dem_vertical_datum": args.dem_vertical_datum,
            "site_cost_model": "zero; no validated tower-construction cost supplied",
        }
        print(f"  [{TOWER_GRAPH_METHOD}] 路径 {len(tower_geo) if tower_geo else 0} 点, "
              f"耗时 {t_tower_graph:.1f}s；不参与最终选路")

    station_offsets_by_method = {}
    station_measurements = {}
    hard_violation_reports = {}
    tower_span_reports = {}
    if station_task:
        for method, geo in list(methods.items()):
            if not geo:
                continue
            geo, offsets = attach_station_endpoints(geo, station_task)
            if method == TOWER_GRAPH_METHOD:
                geo, tower_span_reports[method] = validate_tower_span_lengths(
                    geo, args.tower_min_span_m, args.tower_max_span_m)
                grade_report = grade_reports[method]
            else:
                geo, grade_report = filter_path_by_ground_grade(
                    geo, dem, transform, args.max_ground_grade, crs)
            validation_mask = (tower_span_mask if method == TOWER_GRAPH_METHOD
                               else hard_mask)
            geo, hard_report = validate_route_hard_constraints(
                geo, transform, validation_mask)
            hard_report.update(tower_span_reports.get(method, {}))
            if geo and osm_layer_metadata.get("buildings", {}).get("status") == "available":
                geo, building_report = validate_route_against_buildings(
```

源程序鉴别材料 • 中间程序略，以下为末尾连续部分 1/30

```
                        positive_weight_shapes, out_shape=(height, width), transform=transform,
                        fill=0, all_touched=True, dtype=np.uint8,
                    )
                    affected_cell_count = int(np.count_nonzero(affected))
                else:
                    affected_cell_count = 0
            source_layers.append({
                "source": source,
                "mode": mode,
                "feature_count": len(geometries),
                "covered_cell_count": covered_cell_count,
                "affected_cell_count": affected_cell_count,
            })
    metadata = {
        "path": str(path.resolve()),
        "status": "loaded",
        "feature_count": len(document["features"]),
        "hard_feature_count": len(hard_shapes),
        "soft_feature_count": len(soft_shapes),
        "hard_constraint_evidence": hard_constraint_evidence,
        "hard_constraint_declarations_complete": bool(hard_shapes) and all(
            item["verification_status"] == "verified"
            and item["verification_reference"]
            for item in hard_constraint_evidence),
        "hard_constraint_evidence_validation_scope": "caller_declarations_only",
        "sources": sorted(sources),
        "categories": sorted(categories),
        "hard_cell_count": int(hard_mask.sum()),
        "soft_cell_count": int(np.count_nonzero(soft_cost)),
        "source_layers": source_layers,
    }
    if dataset_metadata is not None:
        metadata["dataset_metadata"] = dataset_metadata
    if data_sources is not None:
        metadata["data_sources"] = data_sources
    return {"hard_mask": hard_mask, "soft_cost": soft_cost, "metadata": metadata}


def compute_vegetation_height(geoms, transform, shape):
    """从植被/林地类 OSM 多边形推导植被高度代理 (m)。"""
    result = np.zeros(shape, np.float32)
    if not geoms:
        return result
    from collections import defaultdict
    groups = defaultdict(list)
    for g, tags in geoms:
        lu = (tags.get("landuse") or "").lower()
        nat = (tags.get("natural") or "").lower()
        if "wood" in nat or "forest" in nat or "tree" in nat:
            h = 20.0
```

源程序鉴别材料 • 末尾连续部分 2/30

```
        elif "scrub" in nat or "grass" in nat or "grassland" in nat or "heath" in nat:
            h = 5.0
        elif "wetland" in nat or "marsh" in nat:
            h = 2.0
        elif "forest" in lu or "wood" in lu:
            h = 20.0
        else:
            h = 3.0
        groups[h].append(g)
    for h, glist in groups.items():
        m = rasterize_on([(g, {}) for g in glist], transform, shape)
        result = np.where(m > 0, h, result).astype(np.float32)
    return result


def compute_osm_bands(dem, transform, bbox, cache_dir=None, *,
                      include_model_features=True, include_building_distance=True):
    """计算 OSM 波段与风险代理；图层状态可从返回值的 ``metadata`` 属性读取。

    Building footprints provide a hard footprint mask and settlement proxy;
    residential land use provides a soft settlement proxy and model feature.
    Other layers are model features only and are not used by the default rule
    cost. Protected areas are reported for inspection only.
    """
    H, W = dem.shape
    cache_dir = Path(cache_dir) if cache_dir else DOWNLOADED_DIR
    layer_specs = {
        "roads": ("taiwan_roads.pkl", "line", "feature_model_input_only_not_default_rule_cost"),
        "water": ("taiwan_water.pkl", "line", "feature_model_input_only_not_default_rule_cost"),
        "railways": ("taiwan_railways.pkl", "line", "feature_model_input_only_not_default_rule_cost"),
        "faults": ("taiwan_faults.pkl", "line", "feature_model_input_only_not_default_rule_cost"),
        "landuse": ("taiwan_landuse.pkl", "polygon", "settlement_and_optional_landcover_soft_proxy"),
        "buildings": ("taiwan_buildings.pkl", "polygon", "hard_footprint_and_settlement_proxy"),
        "vegetation": ("taiwan_vegetation.pkl", "polygon", "feature_model_input_only_not_default_rule_cost"),
        "protected_areas": ("taiwan_protected.pkl", "polygon", "informational_only"),
    }
    layers = {}
    layer_metadata = {}
    for name, (filename, geom_type, role) in layer_specs.items():
        if not include_model_features and name not in ("landuse", "buildings"):
            layers[name] = []
            layer_metadata[name] = {
                "path": str(cache_dir / filename), "status": "not_loaded",
                "reason": "not_required_by_selected_planner_mode",
                "coverage_status": "not_checked", "planning_role": role,
            }
            continue
        query_bbox = bbox
        if name == "buildings":
            latitude = 0.5 * (bbox[1] + bbox[3])
```

源程序鉴别材料 • 末尾连续部分 3/30

```
            buffer_m = 5000.0 if include_building_distance else 2000.0
            lat_buffer = buffer_m / 111320.0
            lon_buffer = lat_buffer / max(math.cos(math.radians(latitude)), 1e-6)
            query_bbox = (bbox[0] - lon_buffer, bbox[1] - lat_buffer,
                          bbox[2] + lon_buffer, bbox[3] + lat_buffer)
        layers[name], layer_metadata[name] = load_osm_geoms_with_status(
            cache_dir / filename, geom_type, query_bbox)
        layer_metadata[name]["planning_role"] = role
    roads = layers["roads"]
    water = layers["water"]
    railways = layers["railways"]
    faults = layers["faults"]
    landuse = layers["landuse"]
    buildings = layers["buildings"]
    vegetation = layers["vegetation"]
    if include_model_features:
        print(f"  OSM 几何数: 道路{len(roads)} 水域{len(water)} 铁路{len(railways)} "
              f"断裂{len(faults)} 土地利用{len(landuse)} 建筑{len(buildings)} 植被{len(vegetation)}")
    else:
        print(f"  OSM 规划图层: 土地利用{len(landuse)} 建筑{len(buildings)}; "
              "未加载仅供模型或诊断的可选图层")

    bands = OSMBands(metadata={"layers": layer_metadata},
                     geometries={"buildings": buildings})
    if include_model_features:
        bands["dist_road"] = compute_distance_band(roads, transform, (H, W), 5000.0)
        bands["dist_water"] = compute_distance_band(water, transform, (H, W), 5000.0)
        bands["dist_railway"] = compute_distance_band(railways, transform, (H, W), 5000.0)
        bands["dist_fault"] = compute_distance_band(faults, transform, (H, W), 10000.0)
    bands["landuse_code"] = compute_landuse_code(landuse, transform, (H, W))
    bands["residential_landuse_mask"] = compute_residential_landuse_mask(landuse, transform, (H, W))
    bands["building_density"] = compute_building_density(buildings, transform, (H, W))
    bands["building_mask"] = rasterize_on(buildings, transform, (H, W))
    if include_building_distance:
        bands["building_distance_m"] = compute_distance_band(buildings, transform, (H, W), 5000.0)
    if include_model_features:
        bands["vegetation_height"] = compute_vegetation_height(vegetation, transform, (H, W))

    if include_model_features:
        # 风险代理波段（21-25）— 由地形/位置推导
        z = _fill(dem).astype(np.float32)
        lat_center = transform.f + transform.e * (H / 2.0)  # 窗口纬度中心（修正：之前误用 0）
        row_m, col_m = pixel_spacing_meters(transform, lat_center)
        dy, dx = _terrain_gradient(z, row_m, col_m)
        slope = np.degrees(np.arctan(np.sqrt(dx ** 2 + dy ** 2)))
        aspect = np.degrees(np.arctan2(-dy, dx)) % 360
        elev_n = np.clip(z / 3000.0, 0, 1)
        slope_n = np.clip(slope / 45.0, 0, 1)
        # 台风：东南坡 + 低海拔 + 陡坡
        aspect_score = np.where((aspect >= 45) & (aspect < 225), 1.0, 0.3)
```

源程序鉴别材料 • 末尾连续部分 4/30

```
        bands["typhoon_risk"] = np.clip(0.4 * slope_n + 0.35 * aspect_score + 0.25 * (1 - elev_n), 0, 1).astype(np.float32)
        # 地震：基础 0.6 + 坡度
        bands["seismic_risk"] = np.clip(0.6 + 0.3 * slope_n, 0, 1).astype(np.float32)
        # 滑坡：坡度 + 粗糙度
        rough = np.nan_to_num(ndi.generic_filter(z, np.nanstd, size=3), nan=0.0) if HAS_SCIPY else np.zeros_like(z)
        bands["landslide_risk"] = np.clip(0.5 * slope_n + 0.3 * np.clip(rough / 200.0, 0, 1), 0, 1).astype(np.float32)
        # 覆冰：高海拔
        bands["ice_cover_risk"] = np.clip((z - 2000) / 2000.0, 0, 1).astype(np.float32)
        # 雷击：坡度 + 海拔
        bands["lightning_risk"] = np.clip(0.6 * slope_n + 0.4 * elev_n, 0, 1).astype(np.float32)
    return bands
# ===== transmission_path_planner/provenance.py =====
"""Run provenance for reproducible route-planning benchmarks."""

from datetime import datetime, timezone
import hashlib
from importlib import metadata
import json
import platform
from pathlib import Path
import subprocess
import sys


OSM_CACHE_FILES = {
    "roads": "taiwan_roads.pkl",
    "water": "taiwan_water.pkl",
    "railways": "taiwan_railways.pkl",
    "faults": "taiwan_faults.pkl",
    "landuse": "taiwan_landuse.pkl",
    "buildings": "taiwan_buildings.pkl",
    "vegetation": "taiwan_vegetation.pkl",
    "protected_areas": "taiwan_protected.pkl",
}
DEPENDENCIES = (
    "numpy", "scipy", "pandas", "shapely", "rasterio", "geopandas",
    "pyproj", "pyogrio", "matplotlib", "torch", "osmnx",
)
PROVENANCE_FILENAME = "run_provenance.json"


def _utc_now():
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


def _json_value(value):
    if isinstance(value, Path):
        return str(value)
    if isinstance(value, dict):
        return {str(key): _json_value(item) for key, item in value.items()}
```

源程序鉴别材料 • 末尾连续部分 5/30

```
    if isinstance(value, (list, tuple)):
        return [_json_value(item) for item in value]
    if value is None or isinstance(value, (str, int, float, bool)):
        return value
    return str(value)


def sha256_file(path):
    digest = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def _file_record(role, path):
    path = Path(path).resolve()
    record = {"role": role, "path": str(path)}
    if not path.is_file():
        record["status"] = "missing"
        return record
    try:
        record.update(
            status="hashed",
            size_bytes=path.stat().st_size,
            sha256=sha256_file(path),
        )
    except OSError as exc:
        record.update(status="unreadable", error=str(exc))
    return record


def _arg(args, name, default=None):
    if isinstance(args, dict):
        return args.get(name, default)
    return getattr(args, name, default)


def _planning_input_records(args, dem_path, default_osm_cache_dir, model_weights,
                            has_torch):
    records = []
    for name, role in (
        ("stations_csv", "station_table"),
        ("route_tasks_csv", "route_tasks"),
        ("real_cases_csv", "case_tasks"),
        ("constraint_geojson", "user_constraints"),
        ("land_mask_geojson", "land_mask"),
        ("population_raster", "population_raster"),
    ):
        path = _arg(args, name)
```

源程序鉴别材料 • 末尾连续部分 6/30

```
        if path:
            records.append(_file_record(role, path))
    records.append(_file_record("dem", dem_path))

    cache_dir = Path(_arg(args, "osm_cache_dir") or default_osm_cache_dir)
    include_model_features = bool(
        _arg(args, "include_ai_diagnostics") and has_torch
        and not _arg(args, "no_vin"))
    layer_names = list(OSM_CACHE_FILES)
    if not include_model_features:
        layer_names = ["landuse", "buildings"]
    for layer_name in layer_names:
        records.append(_file_record(
            f"osm_cache:{layer_name}", cache_dir / OSM_CACHE_FILES[layer_name]))

    hazard_dir = Path(_arg(args, "official_hazards_dir"))
    if hazard_dir.is_dir():
        for fixed_name in ("manifest.json", "tiles.sha256"):
            path = hazard_dir / fixed_name
            if path.is_file():
                records.append(_file_record("official_hazard_manifest", path))
        for path in sorted(hazard_dir.rglob("*.pbf")):
            if path.is_file() and not path.is_symlink():
                records.append(_file_record("official_hazard_tile", path))
    else:
        records.append({
            "role": "official_hazards_dir",
            "path": str(hazard_dir.resolve()),
            "status": "missing",
        })

    if _arg(args, "use_costunet") and has_torch:
        for path in model_weights:
            if Path(path).is_file():
                records.append(_file_record("costunet_weights", path))
    return records


def _source_state(repo_path):
    def git(*arguments, cwd=None):
        return subprocess.run(
            ["git", "-C", str(cwd or repo_path), *arguments],
            check=False, capture_output=True, text=True, encoding="utf-8",
            errors="replace",
        )

    try:
        root = git("rev-parse", "--show-toplevel")
        if root.returncode != 0:
            return {"available": False,
```

源程序鉴别材料 • 末尾连续部分 7/30

```
                      "reason": root.stderr.strip() or "not a Git worktree"}
        git_root = Path(root.stdout.strip())
        commit = git("rev-parse", "HEAD", cwd=git_root)
        branch = git("branch", "--show-current", cwd=git_root)
        status = git("status", "--porcelain=v1", "--untracked-files=normal", cwd=git_root)
    except OSError as exc:
        return {"available": False, "reason": str(exc)}
    if commit.returncode != 0 or status.returncode != 0:
        return {"available": False, "reason": "unable to read Git state"}
    entries = [line for line in status.stdout.splitlines() if line]
    package_dir = Path(repo_path).resolve() / "transmission_path_planner"
    if not package_dir.is_dir():
        package_dir = git_root / "transmission_path_planner"
    source_files = []
    if package_dir.is_dir():
        for path in sorted(package_dir.rglob("*.py")):
            if "__pycache__" in path.parts or path.is_symlink():
                continue
            source_files.append({
                "path": path.relative_to(git_root).as_posix(),
                "size_bytes": path.stat().st_size,
                "sha256": sha256_file(path),
            })
    return {
        "available": True,
        "root": str(git_root),
        "commit": commit.stdout.strip(),
        "branch": branch.stdout.strip() or None,
        "dirty": bool(entries),
        "status_porcelain": entries,
        "runtime_source_files": source_files,
    }


def _runtime_info():
    versions = {}
    for package in DEPENDENCIES:
        try:
            versions[package] = metadata.version(package)
        except metadata.PackageNotFoundError:
            versions[package] = None
    return {
        "python_version": sys.version,
        "python_executable": sys.executable,
        "implementation": platform.python_implementation(),
        "platform": platform.platform(),
        "dependencies": versions,
    }
```

源程序鉴别材料 • 末尾连续部分 8/30

```
def _case_records(cases):
    return [
        {
            "case": name,
            "start_latlon": _json_value(start),
            "end_latlon": _json_value(end),
        }
        for name, start, end in cases
    ]


def capture_run_provenance(repo_path, args, cases, dem_path,
                           default_osm_cache_dir, model_weights=(),
                           has_torch=True, effective_runtime=None):
    """Capture run configuration and planning inputs before route generation.

    Reference geometry paths are recorded as configuration only; this function
    never opens, hashes, or walks ``real_dir`` or ``real_shp``.
    """
    configuration = _json_value(vars(args) if hasattr(args, "__dict__") else args)
    return {
        "schema_version": 1,
        "started_at_utc": _utc_now(),
        "command": {"argv": list(sys.argv), "cwd": str(Path.cwd())},
        "configuration": configuration,
        "cases": _case_records(cases),
        "source": _source_state(repo_path),
        "runtime": _runtime_info(),
        "effective_runtime": _json_value(
            effective_runtime or {"has_torch": has_torch}),
        "planning_inputs": _planning_input_records(
            args, dem_path, default_osm_cache_dir, model_weights, has_torch),
        "reference_geometry_policy": {
            "hashed_by_provenance": False,
            "paths_are_configuration_only": True,
        },
    }


def _is_within(path, parent):
    try:
        path.relative_to(parent)
        return True
    except ValueError:
        return False


def _output_records(output_dir, excluded_reference_paths=()):
    root = Path(output_dir).resolve()
    excluded = [Path(path).resolve() for path in excluded_reference_paths if path]
```

源程序鉴别材料 • 末尾连续部分 9/30

```
    records = []
    for path in sorted(root.rglob("*")):
        if not path.is_file() or path.is_symlink() or path.name == PROVENANCE_FILENAME:
            continue
        resolved = path.resolve()
        if any(resolved == item or _is_within(resolved, item) for item in excluded):
            continue
        record = {"path": path.relative_to(root).as_posix()}
        try:
            record.update(size_bytes=path.stat().st_size, sha256=sha256_file(path),
                          status="hashed")
        except OSError as exc:
            record.update(status="unreadable", error=str(exc))
        records.append(record)
    return records


def write_run_provenance(snapshot, output_dir, excluded_reference_paths=()):
    """Write the final manifest after normal outputs exist, hashing each once."""
    document = dict(snapshot)
    document["completed_at_utc"] = _utc_now()
    document["output_directory"] = str(Path(output_dir).resolve())
    document["outputs"] = _output_records(output_dir, excluded_reference_paths)
    document["output_hash_policy"] = (
        "SHA-256 of regular files under output_directory, excluding this manifest "
        "and paths identified as reference inputs")
    path = Path(output_dir) / PROVENANCE_FILENAME
    path.write_text(json.dumps(document, ensure_ascii=False, indent=2) + "\n",
                    encoding="utf-8")
    return path
# ===== transmission_path_planner/rebuild_terrarium_dem.py =====
"""Rebuild an EPSG:4326 DEM from cached AWS Terrarium XYZ tiles."""

import argparse
import math
import os
import tempfile
from pathlib import Path

import numpy as np
import rasterio
from PIL import Image
from rasterio.enums import Resampling
from rasterio.transform import Affine
from rasterio.warp import reproject, transform_bounds
from rasterio.windows import Window

from .terrarium import XYZ_TILE_SIZE, terrarium_rgb_to_elevation, xyz_tile_transform
```

源程序鉴别材料 • 末尾连续部分 10/30

```
def lonlat_to_xyz(lon, lat, zoom):
    n = 2 ** zoom
    lat = min(max(lat, -85.05112878), 85.05112878)
    x = math.floor((lon + 180.0) / 360.0 * n)
    y = math.floor((1.0 - math.asinh(math.tan(math.radians(lat))) / math.pi)
                   * 0.5 * n)
    return x, y


def rebuild_dem(tile_cache, output_path, bbox, zoom=12, overwrite=False):
    """Warp cached Web Mercator tiles to a regular geographic grid.

    ``bbox`` is ``(min_lon, min_lat, max_lon, max_lat)``. The source tile rows
    remain in EPSG:3857 until GDAL reprojects them; they are never interpreted
    as equally spaced latitude rows.
    """
    cache = Path(tile_cache)
    output = Path(output_path)
    if output.exists() and not overwrite:
        raise FileExistsError(f"output already exists: {output}; pass --overwrite to replace it")
    output.parent.mkdir(parents=True, exist_ok=True)

    min_lon, min_lat, max_lon, max_lat = map(float, bbox)
    x0, y0 = lonlat_to_xyz(min_lon, max_lat, zoom)
    x1, y1 = lonlat_to_xyz(max_lon, min_lat, zoom)
    nx, ny = x1 - x0 + 1, y1 - y0 + 1
    tile_paths = {
        (x, y): cache / f"t_{zoom}_{x}_{y}.png"
        for y in range(y0, y1 + 1)
        for x in range(x0, x1 + 1)
    }
    missing = [str(path) for path in tile_paths.values() if not path.is_file()]
    if missing:
        preview = ", ".join(missing[:5])
        raise FileNotFoundError(f"{len(missing)} XYZ tiles are missing: {preview}")

    source_transform = xyz_tile_transform(zoom, x0, y0)
    source_width, source_height = nx * XYZ_TILE_SIZE, ny * XYZ_TILE_SIZE
    source_profile = {
        "driver": "GTiff", "width": source_width, "height": source_height,
        "count": 1, "dtype": "float32", "crs": "EPSG:3857",
        "transform": source_transform, "nodata": -32768.0,
        "compress": "lzw", "tiled": True,
        "blockxsize": XYZ_TILE_SIZE, "blockysize": XYZ_TILE_SIZE,
        "BIGTIFF": "IF_SAFER",
    }

    with tempfile.TemporaryDirectory(prefix="terrarium-warp-", dir=output.parent) as temp_dir:
        source_path = Path(temp_dir) / "terrarium_3857.tif"
        warped_path = Path(temp_dir) / output.name
```

源程序鉴别材料 • 末尾连续部分 11/30

```
        with rasterio.open(source_path, "w", **source_profile) as source:
            for (tile_x, tile_y), tile_path in tile_paths.items():
                with Image.open(tile_path) as tile_image:
                    rgba = np.asarray(tile_image.convert("RGBA"))
                values = terrarium_rgb_to_elevation(rgba)
                invalid = ((rgba[..., 3] == 0) | (values < -100.0) | (values > 9000.0))
                values[invalid] = source_profile["nodata"]
                source.write(values, 1, window=Window(
                    (tile_x - x0) * XYZ_TILE_SIZE,
                    (tile_y - y0) * XYZ_TILE_SIZE,
                    XYZ_TILE_SIZE, XYZ_TILE_SIZE,
                ))
            source.update_tags(
                source="AWS Terrain Tiles Terrarium XYZ cache",
                zoom=str(zoom),
                tile_range=f"x={x0}-{x1};y={y0}-{y1}",
                elevation_encoding="R*256 + G + B/256 - 32768 metres",
                vertical_datum="not verified from the cached PNG files",
            )

        with rasterio.open(source_path) as source:
            left, bottom, right, top = transform_bounds(
                source.crs, "EPSG:4326", *source.bounds, densify_pts=21)
            center_lat = 0.5 * (bottom + top)
            xres = 360.0 / (XYZ_TILE_SIZE * (2 ** zoom))
            yres = xres * math.cos(math.radians(center_lat))
            width = math.ceil((right - left) / xres)
            height = math.ceil((top - bottom) / yres)
            destination_transform = Affine(xres, 0.0, left, 0.0, -yres, top)
            destination_profile = {
                "driver": "GTiff", "width": width, "height": height,
                "count": 1, "dtype": "float32", "crs": "EPSG:4326",
                "transform": destination_transform, "nodata": -32768.0,
                "compress": "lzw", "tiled": True,
                "blockxsize": XYZ_TILE_SIZE, "blockysize": XYZ_TILE_SIZE,
                "BIGTIFF": "IF_SAFER",
            }
            with rasterio.open(warped_path, "w", **destination_profile) as destination:
                reproject(
                    source=rasterio.band(source, 1),
                    destination=rasterio.band(destination, 1),
                    src_transform=source.transform, src_crs=source.crs,
                    src_nodata=source.nodata,
                    dst_transform=destination_transform, dst_crs="EPSG:4326",
                    dst_nodata=destination_profile["nodata"],
                    resampling=Resampling.bilinear, num_threads=2,
                )
                destination.update_tags(
                    source="AWS Terrain Tiles Terrarium XYZ cache",
                    source_crs="EPSG:3857",
```

源程序鉴别材料 • 末尾连续部分 12/30

```
                    source_zoom=str(zoom),
                    source_tile_range=f"x={x0}-{x1};y={y0}-{y1}",
                    georeferencing="Web Mercator XYZ tiles reprojected to EPSG:4326",
                    resampling="bilinear",
                    vertical_datum="not verified from the cached PNG files",
                )

        os.replace(warped_path, output)

    with rasterio.open(output) as result:
        print(f"Wrote {output}: {result.width}x{result.height}, "
              f"EPSG:4326, bounds={tuple(round(v, 6) for v in result.bounds)}")


def main():
    project_dir = Path(__file__).resolve().parents[1]
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--tile-cache", required=True, help="directory containing t_Z_X_Y.png")
    parser.add_argument("--output", default=str(project_dir / "data" / "taiwan_dem_35m_georef_fixed.tif"))
    parser.add_argument("--bbox", nargs=4, type=float, default=(119.4, 21.7, 122.3, 25.5),
                        metavar=("MIN_LON", "MIN_LAT", "MAX_LON", "MAX_LAT"))
    parser.add_argument("--zoom", type=int, default=12)
    parser.add_argument("--overwrite", action="store_true")
    args = parser.parse_args()
    rebuild_dem(args.tile_cache, args.output, args.bbox, args.zoom, args.overwrite)


if __name__ == "__main__":
    main()
# ===== transmission_path_planner/smoothing.py =====
# transmission_path_planner/smoothing.py
"""路径平滑、硬性约束吸附、高程采样。"""

from .config import *
from .geometry import *

def _rdp(points, epsilon):
    """Ramer-Douglas-Peucker 折线简化。

    改用显式栈的迭代实现，避免对长且高度弯曲的路径递归过深触发
    'maximum recursion depth exceeded' (Case 3 PPO 路径点数常达数千，单边反复
    细分时递归深度可超 1000)。迭代版结果与递归版完全一致。
    """
    points = list(points)
    n = len(points)
    if n < 3:
        return points
    keep = [False] * n
    keep[0] = True
    keep[n - 1] = True
```

源程序鉴别材料 • 末尾连续部分 13/30

```
    stack = [(0, n - 1)]
    while stack:
        s_i, e_i = stack.pop()
        if e_i <= s_i + 1:
            continue
        s = np.array(points[s_i], np.float64)
        e = np.array(points[e_i], np.float64)
        lv = e - s
        ll = np.linalg.norm(lv)
        if ll < 1e-12:
            # 退化段: 端点已标记保留, 丢弃内部点(与原递归版行为一致)
            continue
        unit = lv / ll
        dmax, idx = 0.0, s_i + 1
        for i in range(s_i + 1, e_i):
            v = np.array(points[i], np.float64) - s
            proj = np.clip(np.dot(v, unit), 0, ll)
            closest = s + proj * unit
            d = np.linalg.norm(np.array(points[i], np.float64) - closest)
            if d > dmax:
                dmax, idx = d, i
        if dmax > epsilon:
            keep[idx] = True
            stack.append((s_i, idx))
            stack.append((idx, e_i))
    return [points[i] for i in range(n) if keep[i]]


def smooth_path(geo, spacing_m=90.0):
    if len(geo) < 3:
        return geo
    arr = np.array([(lon, lat) for lon, lat in geo], np.float64)
    from pyproj import CRS, Transformer

    lons = np.radians(arr[:, 0])
    lon0 = math.degrees(math.atan2(float(np.sin(lons).mean()), float(np.cos(lons).mean())))
    lat0 = float(np.mean(arr[:, 1]))
    local_crs = CRS.from_proj4(
        f"+proj=aeqd +lat_0={lat0} +lon_0={lon0} +datum=WGS84 +units=m +no_defs"
    )
    project = Transformer.from_crs("EPSG:4326", local_crs, always_xy=True).transform
    unproject = Transformer.from_crs(local_crs, "EPSG:4326", always_xy=True).transform
    metric = np.asarray([project(lon, lat) for lon, lat in arr], dtype=np.float64)
    epsilon_m = min(max(float(spacing_m), 0.0) * 0.5, 50.0)
    simplified = _rdp(metric.tolist(), epsilon_m)
    if len(simplified) < 2:
        return geo
    simplified = np.array(simplified)
    xs, ys = simplified[:, 0], simplified[:, 1]
    seg = np.sqrt(np.diff(xs) ** 2 + np.diff(ys) ** 2)
```

源程序鉴别材料 • 末尾连续部分 14/30

```
    cum = np.concatenate([[0], np.cumsum(seg)])
    total = cum[-1]
    if total < 1e-9:
        return geo
    n = max(int(total / max(float(spacing_m), 1e-6)), len(simplified))
    sd = np.linspace(0, total, n)
    xi = np.interp(sd, cum, xs); yi = np.interp(sd, cum, ys)
    result = [tuple(map(float, unproject(xi[i], yi[i]))) for i in range(n)]
    result[0], result[-1] = tuple(geo[0]), tuple(geo[-1])
    return result


def snap_to_valid(geo, transform, hard_mask, max_offset_m=100.0):
    """把平滑后的 geo 路径中落到硬约束屏蔽格(陡坡/超高/无效 DEM)的点，吸附到最近的合法格中心。
    推理 rollout 只走合法邻居，原始格路径已是 0 违规；但 smooth_path 在地理空间等距重采样时会
    越过硬约束格。吸附距离使用米制上限，超出范围则拒绝候选并由调用方回退到原始路径。"""
    H, W = hard_mask.shape
    center_lat = transform.f + transform.e * (H / 2.0)
    row_m, col_m = pixel_spacing_meters(transform, center_lat)
    max_cells = int(math.ceil(max(float(max_offset_m), 0.0) / min(row_m, col_m)))
    out = []
    for lon, lat in geo:
        r, c = geo_to_rowcol(lat, lon, transform)
        r = int(round(r)); c = int(round(c))
        if 0 <= r < H and 0 <= c < W and hard_mask[r, c] == 1:
            out.append((lon, lat)); continue
        candidates = []
        for dr in range(-max_cells, max_cells + 1):
            for dc in range(-max_cells, max_cells + 1):
                rr, cc = r + dr, c + dc
                if not (0 <= rr < H and 0 <= cc < W) or hard_mask[rr, cc] != 1:
                    continue
                distance_m = haversine_km(lon, lat, *rowcol_to_geo(rr, cc, transform)) * 1000.0
                if distance_m <= max_offset_m:
                    candidates.append((distance_m, rr, cc))
        if not candidates:
            return None
        _, rr, cc = min(candidates)
        out.append(rowcol_to_geo(rr, cc, transform))
    return out


def _grid_cells_touched(c0, r0, c1, r1):
    """Return every raster cell touched by a segment in grid-corner coordinates."""
    dx, dy = c1 - c0, r1 - r0
    cells = set()

    def axis_cells(value):
        nearest = round(value)
        if abs(value - nearest) <= 1e-10:
```

源程序鉴别材料 • 末尾连续部分 15/30

```
            return (int(nearest) - 1, int(nearest))
        return (int(math.floor(value)),)

    def add_touching(x, y):
        for row in axis_cells(y):
            for col in axis_cells(x):
                cells.add((row, col))

    add_touching(c0, r0)
    add_touching(c1, r1)

    sx = 1 if dx > 0 else -1 if dx < 0 else 0
    sy = 1 if dy > 0 else -1 if dy < 0 else 0
    col, row = int(math.floor(c0)), int(math.floor(r0))
    if sx:
        t_max_x = ((col + 1 - c0) if sx > 0 else (c0 - col)) / abs(dx)
        t_delta_x = 1.0 / abs(dx)
    else:
        t_max_x = t_delta_x = math.inf
    if sy:
        t_max_y = ((row + 1 - r0) if sy > 0 else (r0 - row)) / abs(dy)
        t_delta_y = 1.0 / abs(dy)
    else:
        t_max_y = t_delta_y = math.inf

    while min(t_max_x, t_max_y) <= 1.0 + 1e-12:
        if abs(t_max_x - t_max_y) <= 1e-12:
            cells.add((row, col + sx))
            cells.add((row + sy, col))
            col += sx
            row += sy
            cells.add((row, col))
            t_max_x += t_delta_x
            t_max_y += t_delta_y
        elif t_max_x < t_max_y:
            col += sx
            cells.add((row, col))
            t_max_x += t_delta_x
        else:
            row += sy
            cells.add((row, col))
            t_max_y += t_delta_y

    if sx == 0 and abs(c0 - round(c0)) <= 1e-10:
        boundary_col = int(round(c0))
        cells.update((row, boundary_col - 1) for row, col in tuple(cells)
                     if col == boundary_col)
    if sy == 0 and abs(r0 - round(r0)) <= 1e-10:
        boundary_row = int(round(r0))
        cells.update((boundary_row - 1, col) for row, col in tuple(cells)
```

源程序鉴别材料 • 末尾连续部分 16/30

```
                    if row == boundary_row)
    return cells


def path_hard_violations(geo, transform, hard_mask):
    """Count blocked/out-of-grid cells crossed by a path, including segment interiors."""
    if not geo:
        return 0
    inv = ~transform
    cells = set()
    grid_points = [inv * point for point in geo]
    for (c0, r0), (c1, r1) in zip(grid_points, grid_points[1:]):
        cells.update(_grid_cells_touched(c0, r0, c1, r1))
    if len(grid_points) == 1:
        c, r = grid_points[0]
        cells.update(_grid_cells_touched(c, r, c, r))
    return sum(1 for r, c in cells
               if not (0 <= r < hard_mask.shape[0] and 0 <= c < hard_mask.shape[1])
               or hard_mask[r, c] == 0)


def _ground_grade_segment_cells(c0, r0, c1, r1, dem):
    """Sample the DEM cells along one segment using the final route validator's rule."""
    H, W = dem.shape
    steps = max(1, int(math.ceil(4 * max(abs(r1 - r0), abs(c1 - c0)))))
    cells = []
    for i in range(steps + 1):
        t = i / steps
        r, c = r0 + (r1 - r0) * t, c0 + (c1 - c0) * t
        rr, cc = int(math.floor(r)), int(math.floor(c))
        cell = ((rr, cc) if 0 <= rr < H and 0 <= cc < W
                and np.isfinite(dem[rr, cc]) else None)
        if not cells or cell != cells[-1]:
            cells.append(cell)
    return cells


def _ground_grade_cells_report(cells, dem, row_spacing_m, col_spacing_m,
                               max_ground_grade):
    violations = 0
    unassessed = 0
    max_ratio = 0.0
    for first, second in zip(cells, cells[1:]):
        if first is None or second is None:
            unassessed += 1
            continue
        r0, c0 = first
        r1, c1 = second
        if first == second:
            continue
```

源程序鉴别材料 • 末尾连续部分 17/30

```
        distance_m = math.hypot((r1 - r0) * row_spacing_m,
                                (c1 - c0) * col_spacing_m)
        if distance_m <= 0:
            unassessed += 1
            continue
        ratio = abs(float(dem[r1, c1]) - float(dem[r0, c0])) / distance_m
        max_ratio = max(max_ratio, ratio)
        if ratio > max_ground_grade + 1e-9:
            violations += 1
    return violations, unassessed, max_ratio


def _ground_grade_grid_edge_passes(start_rc, end_rc, dem, pixel_spacing,
                                   max_ground_grade):
    """Check a cell-center A* edge with the same DEM sampling as route validation."""
    row0, col0 = start_rc
    row1, col1 = end_rc
    cells = _ground_grade_segment_cells(
        col0 + 0.5, row0 + 0.5, col1 + 0.5, row1 + 0.5, dem)
    violations, unassessed, _ = _ground_grade_cells_report(
        cells, dem, pixel_spacing[0], pixel_spacing[1], max_ground_grade)
    return violations == 0 and unassessed == 0


def ground_grade_limit_report(geo, dem, transform, max_ground_grade, crs=None):
    """Check consecutive DEM cells crossed by a route against a ground-grade ratio cap."""
    if max_ground_grade is None:
        return {
            "grade_limit_status": "not_configured",
            "grade_limit_violation_count": 0,
            "grade_limit_unassessed_count": 0,
            "grade_limit_max_sampled_grade_ratio": None,
            "grade_limit_threshold_ratio": None,
        }
    if not geo or len(geo) < 2:
        return {
            "grade_limit_status": "no_path",
            "grade_limit_violation_count": 0,
            "grade_limit_unassessed_count": 0,
            "grade_limit_max_sampled_grade_ratio": None,
            "grade_limit_threshold_ratio": max_ground_grade,
        }

    inv = ~transform
    lat_center = transform.f + transform.e * (dem.shape[0] / 2.0)
    row_spacing_m, col_spacing_m = pixel_spacing_meters(transform, lat_center, crs)
    cells = []
    last_cell = object()
    grid_points = [inv * point for point in geo]
    if len(grid_points) == 1:
```

源程序鉴别材料 • 末尾连续部分 18/30

```
        segments = [(grid_points[0], grid_points[0])]
    else:
        segments = zip(grid_points, grid_points[1:])
    for (c0, r0), (c1, r1) in segments:
        for marker in _ground_grade_segment_cells(c0, r0, c1, r1, dem):
            if marker != last_cell:
                cells.append(marker)
                last_cell = marker

    violations, unassessed, max_ratio = _ground_grade_cells_report(
        cells, dem, row_spacing_m, col_spacing_m, max_ground_grade)

    status = "fail" if violations or unassessed else "pass"
    return {
        "grade_limit_status": status,
        "grade_limit_violation_count": violations,
        "grade_limit_unassessed_count": unassessed,
        "grade_limit_max_sampled_grade_ratio": round(max_ratio, 6),
        "grade_limit_threshold_ratio": max_ground_grade,
    }


def filter_path_by_ground_grade(geo, dem, transform, max_ground_grade, crs=None):
    """Reject a route when a configured grade cap is violated or cannot be assessed."""
    report = ground_grade_limit_report(geo, dem, transform, max_ground_grade, crs)
    if max_ground_grade is not None and report["grade_limit_status"] != "pass":
        return None, report
    return geo, report


def smooth_path_with_constraints(geo, transform, hard_mask, cost_surface=None, dem=None,
                                 grade_penalty_weight=1.0, turn_penalty_weight=0.0,
                                 pixel_spacing=None, max_ground_grade=None, crs=None,
                                 shortcut_endpoints=None):
    """Try a legal, non-increasing direct shortcut, then use the existing smoother."""
    if not geo:
        return None

    def grade_is_valid(path):
        return (max_ground_grade is None or (dem is not None and
                ground_grade_limit_report(
                    path, dem, transform, max_ground_grade, crs
                )["grade_limit_status"] == "pass"))

    if len(geo) >= 3 and cost_surface is not None and dem is not None:
        direct = [geo[0], geo[-1]]
        raw_for_guard = list(geo)
        direct_for_guard = direct
        if shortcut_endpoints is not None:
            start, end = shortcut_endpoints
```

源程序鉴别材料 • 末尾连续部分 19/30

```
            start = tuple(map(float, start))
            end = tuple(map(float, end))
            raw_for_guard[0], raw_for_guard[-1] = start, end
            direct_for_guard = [start, end]
        if path_hard_violations(direct_for_guard, transform, hard_mask) == 0:
            grade_routes = (
                [direct] if direct_for_guard == direct else [direct, direct_for_guard]
            )
            grade_ok = max_ground_grade is None or all(
                ground_grade_limit_report(
                    route, dem, transform, max_ground_grade, crs
                )["grade_limit_status"] == "pass"
                for route in grade_routes
            )
            if grade_ok:
                # Import at call time to avoid the metrics -> smoothing import cycle.
                from .metrics import path_cost_integral

                raw_objective = path_cost_integral(
                    raw_for_guard, dem, transform, cost_surface, grade_penalty_weight,
                    turn_penalty_weight, pixel_spacing)
                direct_objective = path_cost_integral(
                    direct_for_guard, dem, transform, cost_surface, grade_penalty_weight,
                    turn_penalty_weight, pixel_spacing)
                if (raw_objective.get("status") == "measured"
                        and direct_objective.get("status") == "measured"):
                    raw_cost = raw_objective["total_cost_meter"]
                    direct_cost = direct_objective["total_cost_meter"]
                    tolerance = max(abs(raw_cost), 1.0) * 1e-9
                    if direct_cost <= raw_cost + tolerance:
                        return direct

    candidate = snap_to_valid(smooth_path(geo), transform, hard_mask)
    if (candidate and path_hard_violations(candidate, transform, hard_mask) == 0
            and grade_is_valid(candidate)):
        if cost_surface is None:
            return candidate
        if dem is not None:
            # Import at call time to avoid the metrics -> smoothing import cycle.
            from .metrics import path_cost_integral

            raw_objective = path_cost_integral(
                geo, dem, transform, cost_surface, grade_penalty_weight,
                turn_penalty_weight, pixel_spacing)
            smooth_objective = path_cost_integral(
                candidate, dem, transform, cost_surface, grade_penalty_weight,
                turn_penalty_weight, pixel_spacing)
            if (raw_objective.get("status") == "measured"
                    and smooth_objective.get("status") == "measured"):
                raw_cost = raw_objective["total_cost_meter"]
```

源程序鉴别材料 • 末尾连续部分 20/30

```
                smooth_cost = smooth_objective["total_cost_meter"]
                tolerance = max(abs(raw_cost), 1.0) * 1e-9
                if smooth_cost <= raw_cost + tolerance:
                    return candidate
    if (path_hard_violations(geo, transform, hard_mask) == 0
            and grade_is_valid(geo)):
        return geo
    return None


def sample_dem(coords, dem, transform):
    out = []
    H, W = dem.shape
    for lon, lat in coords:
        r, c = geo_to_rowcol(lat, lon, transform)
        if 0 <= r < H and 0 <= c < W and np.isfinite(dem[r, c]):
            out.append(float(dem[r, c]))
        else:
            out.append(np.nan)
    return out


def densify_path_for_raster(coords, transform):
    """Interpolate each route segment so no step exceeds one pixel on either axis."""
    coords = list(coords or [])
    if len(coords) < 2:
        return coords
    inv = ~transform
    grid = [inv * point for point in coords]
    dense = [coords[0]]
    for i, ((c0, r0), (c1, r1)) in enumerate(zip(grid, grid[1:])):
        steps = max(1, int(math.ceil(max(abs(r1 - r0), abs(c1 - c0)))))
        x0, y0 = coords[i]
        x1, y1 = coords[i + 1]
        dense.extend((x0 + (x1 - x0) * step / steps,
                      y0 + (y1 - y0) * step / steps) for step in range(1, steps + 1))
    return dense


def cumulative_relief(hs):
    hs = np.array([h for h in hs if np.isfinite(h)], np.float64)
    if hs.size < 2:
        return 0.0, 0.0
    return float(np.sum(np.abs(np.diff(hs)))), float(hs[-1] - hs[0])
# ===== transmission_path_planner/stations.py =====
"""站点台账与站间线路任务输入。"""

import csv
import hashlib
import math
```

源程序鉴别材料 • 末尾连续部分 21/30

```
import re


def route_output_name(route_id):
    """Return a filesystem-safe, collision-resistant directory name for a route ID."""
    route_id = str(route_id)
    stem = re.sub(r"[^A-Za-z0-9_-]+", "_", route_id).strip("_-")[:64] or "route"
    suffix = hashlib.sha256(route_id.encode("utf-8")).hexdigest()[:10]
    return f"{stem}-{suffix}"


def station_coordinate_crs_properties(station_task):
    if not station_task:
        return {}
    return {
        "from_station_coordinate_crs": station_task["from_station"]["coordinate_crs"],
        "to_station_coordinate_crs": station_task["to_station"]["coordinate_crs"],
    }


def station_endpoint_properties(station_task):
    if not station_task:
        return {}
    result = {"station_endpoint_qualification": station_task["endpoint_qualification"]}
    for prefix, key in (("from", "from_station"), ("to", "to_station")):
        station = station_task[key]
        result.update({
            f"{prefix}_station_point_role": station["point_role"],
            f"{prefix}_station_verification_status": station["verification_status"],
            f"{prefix}_station_source": station["source"] or None,
            f"{prefix}_station_verification_reference": station["verification_reference"] or None,
        })
    return result


def station_elevation_metrics(station_task):
    """Summarize surveyed station-ground elevations without mixing vertical datums."""
    start = station_task["from_station"]
    end = station_task["to_station"]
    start_elevation = start["ground_elevation_m"]
    end_elevation = end["ground_elevation_m"]
    start_datum = start["vertical_datum"]
    end_datum = end["vertical_datum"]
    if start_elevation is None and end_elevation is None:
        status, delta = "dem_only", None
    elif start_elevation is None or end_elevation is None:
        status, delta = "incomplete_station_measurements", None
    elif not start["ground_elevation_source"] or not end["ground_elevation_source"]:
        status, delta = "elevation_source_missing", None
    elif not start_datum or not end_datum:
```

源程序鉴别材料 • 末尾连续部分 22/30

```
        status, delta = "vertical_datum_missing", None
    elif start_datum.casefold() != end_datum.casefold():
        status, delta = "incompatible_vertical_datums", None
    else:
        status = "comparable"
        delta = round(end_elevation - start_elevation, 2)
    return {
        "from_station_ground_elevation_m": start_elevation,
        "from_station_ground_elevation_source": start["ground_elevation_source"] or None,
        "from_station_vertical_datum": start_datum or None,
        "to_station_ground_elevation_m": end_elevation,
        "to_station_ground_elevation_source": end["ground_elevation_source"] or None,
        "to_station_vertical_datum": end_datum or None,
        "station_ground_elevation_status": status,
        "station_ground_elevation_delta_m": delta,
    }


def station_clearance_elevation_overrides(station_task, dem_vertical_datum=None):
    """Return surveyed terminal elevations only when their declared datum matches the DEM."""
    if not station_task:
        return None, "dem_only"
    start = station_task["from_station"]
    end = station_task["to_station"]
    if start["ground_elevation_m"] is None and end["ground_elevation_m"] is None:
        return None, "dem_only"

    metrics = station_elevation_metrics(station_task)
    if metrics["station_ground_elevation_status"] != "comparable":
        return None, metrics["station_ground_elevation_status"]
    if not dem_vertical_datum or not str(dem_vertical_datum).strip():
        return None, "dem_vertical_datum_missing"
    station_datum = start["vertical_datum"].strip()
    if station_datum.casefold() != str(dem_vertical_datum).strip().casefold():
        return None, "incompatible_dem_vertical_datum"
    return {
        "start_ground_elevation_m": start["ground_elevation_m"],
        "goal_ground_elevation_m": end["ground_elevation_m"],
    }, "station_and_dem_vertical_datums_declared_matching"


def _dict_reader(file_obj, label, required_fields):
    reader = csv.DictReader(file_obj)
    fields = reader.fieldnames or []
    if any(not field for field in fields) or len(fields) != len(set(fields)):
        raise ValueError(f"{label}表头存在空列名或重复列名")
    missing = required_fields - set(fields)
    if missing:
        raise ValueError(f"{label}缺少列: {', '.join(sorted(missing))}")
    return reader
```

源程序鉴别材料 • 末尾连续部分 23/30

```
def load_station_tasks(stations_csv, route_tasks_csv, allow_unverified=False):
    """读取 stations.csv 与 route_tasks.csv，验证站点引用并返回规划任务。"""
    stations = {}
    with open(stations_csv, encoding="utf-8-sig", newline="") as f:
        reader = _dict_reader(f, "站点", {"station_id", "lat", "lon", "coordinate_crs"})
        for row_number, row in enumerate(reader, start=2):
            if None in row:
                raise ValueError(f"站点表第 {row_number} 行字段数多于表头")
            station_id = (row.get("station_id") or "").strip()
            if not station_id:
                raise ValueError("站点表缺少 station_id")
            if station_id in stations:
                raise ValueError(f"站点 ID 重复: {station_id}")
            coordinate_crs = (row.get("coordinate_crs") or "").strip().upper()
            if coordinate_crs != "EPSG:4326":
                raise ValueError(
                    f"站点 {station_id} 的 coordinate_crs 必须明确为 EPSG:4326；"
                    "其他坐标系请先转换后再填写 lat/lon")
            try:
                lat, lon = float(row["lat"]), float(row["lon"])
            except (KeyError, TypeError, ValueError) as exc:
                raise ValueError(f"站点 {station_id} 的 lat/lon 无效") from exc
            if not (-90 <= lat <= 90 and -180 <= lon <= 180):
                raise ValueError(f"站点 {station_id} 的经纬度超出范围")
            elevation_text = (row.get("ground_elevation_m") or "").strip()
            try:
                ground_elevation = float(elevation_text) if elevation_text else None
            except ValueError as exc:
                raise ValueError(f"站点 {station_id} 的 ground_elevation_m 无效") from exc
            if ground_elevation is not None and not math.isfinite(ground_elevation):
                raise ValueError(f"站点 {station_id} 的 ground_elevation_m 必须是有限数值")
            point_role = (row.get("point_role") or "unknown").strip().casefold()
            verification_status = (row.get("verification_status") or "unknown").strip().casefold()
            if point_role not in {"line_connection_point", "station_centroid", "unknown"}:
                raise ValueError(f"站点 {station_id} 的 point_role 无效")
            if verification_status not in {"verified", "unverified", "unknown"}:
                raise ValueError(f"站点 {station_id} 的 verification_status 无效")
            stations[station_id] = {
                "station_id": station_id,
                "name": (row.get("name") or station_id).strip(),
                "lat": lat,
                "lon": lon,
                "coordinate_crs": coordinate_crs,
                "voltage_kv": (row.get("voltage_kv") or "").strip(),
                "station_type": (row.get("station_type") or "").strip(),
                "source": (row.get("source") or "").strip(),
                "verified": (row.get("verified") or "").strip(),
                "point_role": point_role,
```

源程序鉴别材料 • 末尾连续部分 24/30

```
                "verification_status": verification_status,
                "verification_reference": (row.get("verification_reference") or "").strip(),
                "ground_elevation_m": ground_elevation,
                "ground_elevation_source": (row.get("ground_elevation_source") or "").strip(),
                "vertical_datum": (row.get("vertical_datum") or "").strip(),
            }

    tasks = []
    route_ids = set()
    with open(route_tasks_csv, encoding="utf-8-sig", newline="") as f:
        reader = _dict_reader(
            f, "线路任务", {"route_id", "from_station_id", "to_station_id"})
        for row_number, row in enumerate(reader, start=2):
            if None in row:
                raise ValueError(f"线路任务表第 {row_number} 行字段数多于表头")
            route_id = (row.get("route_id") or "").strip()
            from_id = (row.get("from_station_id") or "").strip()
            to_id = (row.get("to_station_id") or "").strip()
            if not route_id or not from_id or not to_id:
                raise ValueError("线路任务表需要 route_id、from_station_id、to_station_id")
            if route_id in route_ids:
                raise ValueError(f"线路任务 ID 重复: {route_id}")
            if from_id == to_id:
                raise ValueError(f"线路任务起终站相同: {route_id}")
            if from_id not in stations or to_id not in stations:
                raise ValueError(f"线路任务 {route_id} 引用了站点表中不存在的 ID")
            start, end = stations[from_id], stations[to_id]
            route_voltage = (row.get("voltage_kv") or "").strip()
            if not route_voltage:
                if start["voltage_kv"] and end["voltage_kv"] and start["voltage_kv"] != end["voltage_kv"]:
                    raise ValueError(f"线路任务 {route_id} 两端电压等级不同，请在任务表填写线路 voltage_kv")
                route_voltage = start["voltage_kv"] or end["voltage_kv"]
            endpoints_declared_verified = all(
                station["point_role"] == "line_connection_point"
                and station["verification_status"] == "verified"
                and station["source"]
                and station["verification_reference"]
                for station in (start, end)
            )
            endpoint_qualification = (
                "declared_verified_line_connection_points"
                if endpoints_declared_verified else "exploratory_only_unverified_endpoints"
            )
            if not endpoints_declared_verified and not allow_unverified:
                raise ValueError(
                    f"线路任务 {route_id} 的两端必须声明为已核验线路接入点，"
                    "并填写 source 与 verification_reference；探索用途可显式使用 "
                    "--allow-unverified-endpoints"
                )
            tasks.append({
```

源程序鉴别材料 • 末尾连续部分 25/30

```
                "route_id": route_id,
                "from_station": start,
                "to_station": end,
                "start": (start["lat"], start["lon"]),
                "end": (end["lat"], end["lon"]),
                "voltage_kv": route_voltage,
                "endpoint_qualification": endpoint_qualification,
            })
            route_ids.add(route_id)
    if not tasks:
        raise ValueError("线路任务表为空")
    return tasks
# ===== transmission_path_planner/terrarium.py =====
"""Helpers for correctly locating XYZ Terrarium elevation pixels."""

import math

import numpy as np
from affine import Affine


WEB_MERCATOR_RADIUS_M = 6378137.0
XYZ_TILE_SIZE = 256


def terrarium_rgb_to_elevation(rgb):
    """Decode Terrarium RGB pixels into metres, preserving float fractions."""
    pixels = np.asarray(rgb)
    if pixels.ndim < 3 or pixels.shape[-1] < 3:
        raise ValueError("Terrarium pixels must have at least three RGB channels")
    values = pixels[..., :3].astype(np.float64, copy=False)
    return (values[..., 0] * 256.0 + values[..., 1]
            + values[..., 2] / 256.0 - 32768.0).astype(np.float32)


def xyz_tile_transform(zoom, tile_x, tile_y):
    """Return the source-pixel affine transform in EPSG:3857 for an XYZ tile."""
    scale = XYZ_TILE_SIZE * (2 ** int(zoom))
    resolution = 2.0 * math.pi * WEB_MERCATOR_RADIUS_M / scale
    left = -math.pi * WEB_MERCATOR_RADIUS_M + tile_x * XYZ_TILE_SIZE * resolution
    top = math.pi * WEB_MERCATOR_RADIUS_M - tile_y * XYZ_TILE_SIZE * resolution
    return Affine(resolution, 0.0, left, 0.0, -resolution, top)


def xyz_pixel_to_lonlat(zoom, tile_x, tile_y, pixel_x, pixel_y):
    """Return the WGS84 center of a pixel in a Web Mercator XYZ tile."""
    x, y = xyz_tile_transform(zoom, tile_x, tile_y) * (pixel_x + 0.5, pixel_y + 0.5)
    lon = math.degrees(x / WEB_MERCATOR_RADIUS_M)
    lat = math.degrees(math.atan(math.sinh(y / WEB_MERCATOR_RADIUS_M)))
    return lon, lat
```

源程序鉴别材料 • 末尾连续部分 26/30

```
# ===== transmission_path_planner/visualization.py =====
# transmission_path_planner/visualization.py
"""结果可视化（规划路线对比、价值图、指标柱状图）。"""

from .config import *
from .geometry import *
from .data import _hillshade, _fill
from .io_export import *


def _block_mean_for_display(values, factor):
    values = np.asarray(values)
    if factor <= 1:
        return values
    height, width = values.shape
    out_h, out_w = math.ceil(height / factor), math.ceil(width / factor)
    padded = np.pad(
        values,
        ((0, out_h * factor - height), (0, out_w * factor - width)),
        mode="edge",
    )
    return padded.reshape(out_h, factor, out_w, factor).mean(axis=(1, 3))


def _block_max_for_display(mask, factor):
    mask = np.asarray(mask)
    if factor <= 1:
        return mask
    height, width = mask.shape
    out_h, out_w = math.ceil(height / factor), math.ceil(width / factor)
    padded = np.pad(
        mask.astype(np.uint8),
        ((0, out_h * factor - height), (0, out_w * factor - width)),
        mode="constant",
        constant_values=0,
    )
    return padded.reshape(out_h, factor, out_w, factor).max(axis=(1, 3))


def plot_comparison(dem, transform, methods_paths, start_ll, end_ll, out_path, title="规划路线对比",
                    real_ll=None, hazard_masks=None):
    H, W = dem.shape
    left = transform.c; bottom = transform.f + transform.e * H
    right = transform.c + transform.a * W; top = transform.f
    extent = [left, right, bottom, top]
    fig, ax = plt.subplots(figsize=(12, 9), dpi=110)
    try:
        factor = max(1, math.ceil(max(H, W) / 1200))
        display_dem = _block_mean_for_display(dem, factor)
        hill = _hillshade(display_dem)
```

源程序鉴别材料 • 末尾连续部分 27/30

```
        ax.imshow(hill, extent=extent, cmap="gray", origin="upper", aspect="auto")
        z = _fill(display_dem).astype(np.float32, copy=False)
        ax.imshow(z, extent=extent, cmap="terrain", origin="upper", aspect="auto",
                  alpha=0.30, vmin=np.nanpercentile(z, 2), vmax=np.nanpercentile(z, 98))
        hazard_colors = {
            "debris_flow_impact": ("#f39c12", "土石流影响范围"),
            "landslide_potential": ("#9b59b6", "大规模崩塌潜势区"),
            "landslide_impact": ("#e74c3c", "大规模崩塌影响范围"),
        }
        hazard_legend = None
        if hazard_masks:
            from matplotlib.colors import ListedColormap
            from matplotlib.patches import Patch
            handles = []
            for key, (color, label) in hazard_colors.items():
                mask = hazard_masks.get(key)
                if mask is None:
                    continue
                mask = np.asarray(mask)
                if factor > 1:
                    mask = _block_max_for_display(mask, factor)
                if not mask.any():
                    continue
                ax.imshow(np.ma.masked_where(mask == 0, mask),
                          extent=extent, cmap=ListedColormap([color]),
                          origin="upper", aspect="auto", alpha=0.28,
                          interpolation="nearest", zorder=2)
                handles.append(Patch(facecolor=color, alpha=0.28, label=label))
            if handles:
                hazard_legend = ax.legend(handles=handles, loc="lower left", framealpha=0.85,
                                          title="官方灾害面（软风险）")
        colors = {"VIN-Grad": "#1f77ff", "PPO": "#2ca02c", "A*(baseline)": "#ff7f0e",
                  "A*(weighted)": "#9467bd", "A*(turn-aware)": "#17becf",
                  "A*(8-neighbor exact experimental)": "#8c564b",
                  "A*(16-neighbor experimental)": "#7f7f7f",
                  "A*(8-neighbor shared objective experimental)": "#bcbd22",
                  "A*(16-neighbor shared objective experimental)": "#d62728",
                  "Theta*(experimental)": "#e377c2"}
        line_styles = {
            "A*(baseline)": "-",
            "A*(weighted)": "--",
            "A*(turn-aware)": ":",
            "A*(8-neighbor exact experimental)": "-.",
            "A*(16-neighbor experimental)": (0, (3, 1, 1, 1)),
            "A*(8-neighbor shared objective experimental)": (0, (1, 1)),
            "A*(16-neighbor shared objective experimental)": (0, (5, 2, 1, 2)),
            "Theta*(experimental)": (0, (5, 1)),
        }
        for name, geo in methods_paths.items():
            if not geo:
```

源程序鉴别材料 • 末尾连续部分 28/30

```
                continue
            ax.plot([p[0] for p in geo], [p[1] for p in geo],
                    color=colors.get(name, "#555"),
                    linestyle=line_styles.get(name, "-"),
                    linewidth=2.2, label=f"{name} ({path_length_km(geo):.1f}km)", zorder=4)
        if real_ll and len(real_ll) >= 1:
            from matplotlib.collections import LineCollection
            polys = real_ll if isinstance(real_ll[0][0], (list, tuple)) else [real_ll]
            segs = [[(p[0], p[1]) for p in poly] for poly in polys if len(poly) >= 2]
            if segs:
                lc = LineCollection(segs, colors="black", linestyles=":", linewidths=2.2, zorder=6)
                ax.add_collection(lc)
                ax.plot([], [], color="black", linestyle=":", linewidth=2.6,
                        label=f"配对参考线 ({real_length_km(real_ll):.0f}km)", zorder=6)
        if start_ll:
            ax.scatter([start_ll[1]], [start_ll[0]], c="green", s=130, marker="o", edgecolors="k", label="起点", zorder=5)
        if end_ll:
            ax.scatter([end_ll[1]], [end_ll[0]], c="red", s=150, marker="*", edgecolors="k", label="终点", zorder=5)
        ax.set_xlabel("Longitude"); ax.set_ylabel("Latitude")
        ax.set_title(title)
        ax.legend(loc="upper right", framealpha=0.9)
        if hazard_legend:
            ax.add_artist(hazard_legend)
        ax.set_aspect("equal", adjustable="datalim")
        fig.tight_layout()
        fig.savefig(out_path, dpi=110)
    finally:
        plt.close(fig)
    print(f"  对比图保存: {out_path}")


def plot_value_map(V, transform, geo_vin, start_ll, end_ll, out_path):
    H, W = V.shape
    left = transform.c; bottom = transform.f + transform.e * H
    right = transform.c + transform.a * W; top = transform.f
    extent = [left, right, bottom, top]
    fig, ax = plt.subplots(figsize=(11, 9), dpi=160)
    vmin, vmax = np.nanpercentile(V, 1), np.nanpercentile(V, 99)
    im = ax.imshow(V, extent=extent, cmap="viridis", origin="upper", aspect="auto", vmin=vmin, vmax=vmax)
    if geo_vin:
        ax.plot([p[0] for p in geo_vin], [p[1] for p in geo_vin], color="white", linewidth=1.6, zorder=4)
    if start_ll:
        ax.scatter([start_ll[1]], [start_ll[0]], c="lime", s=120, marker="o", edgecolors="k", zorder=5)
    if end_ll:
        ax.scatter([end_ll[1]], [end_ll[0]], c="red", s=150, marker="*", edgecolors="k", zorder=5)
    ax.set_title("神经值传播概率图 V(s) 与梯度追踪路径")
    ax.set_xlabel("Longitude"); ax.set_ylabel("Latitude")
    ax.set_aspect("equal", adjustable="datalim")
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04, label="值(代价-到此为止)")
    fig.tight_layout(); fig.savefig(out_path, dpi=160); plt.close(fig)
```

源程序鉴别材料 • 末尾连续部分 29/30

```
    print(f"  值传播概率图保存: {out_path}")


_COMPARISON_METHODS = [
    "VIN-Grad", "PPO", "A*(baseline)", "A*(weighted)", "A*(turn-aware)",
    "A*(8-neighbor exact experimental)", "A*(16-neighbor experimental)",
    "A*(8-neighbor shared objective experimental)",
    "A*(16-neighbor shared objective experimental)",
    "Theta*(experimental)",
]
_COMPARISON_METRICS = [
    "length_km", "mean_cost", "sinuosity", "hard_violations", "time_s", "cum_relief_m",
]


def _algo_comparison_data(all_results):
    methods = [m for m in _COMPARISON_METHODS
               if any(m in rec["metrics"] for rec in all_results)]
    agg = {m: {k: [] for k in _COMPARISON_METRICS} for m in methods}
    for rec in all_results:
        for m, met in rec["metrics"].items():
            if m not in agg:   # 跳过最终路线记录等非量化对比条目
                continue
            for k in _COMPARISON_METRICS:
                if (k == "hard_violations"
                        and met.get("hard_constraint_status") == "no_path"):
                    continue
                value = met.get(k)
                if value is not None and isinstance(value, (int, float)) and math.isfinite(value):
                    agg[m][k].append(value)
    return methods, agg


def plot_algo_comparison(all_results, out_path):
    """多算法指标聚合柱状图。"""
    methods, agg = _algo_comparison_data(all_results)
    if not methods:
        return
    metrics_keys = _COMPARISON_METRICS
    means = {m: {k: (sum(v) / len(v) if v else None)
                 for k, v in agg[m].items()} for m in methods}
    short_names = {
        "A*(baseline)": "A* 基线", "A*(weighted)": "A* 加权",
        "A*(turn-aware)": "A* 转向代价", "A*(8-neighbor exact experimental)": "A* 8 邻域精确",
        "A*(16-neighbor experimental)": "A* 16 邻域", "Theta*(experimental)": "Theta*",
        "A*(8-neighbor shared objective experimental)": "A* 8 邻域共目标",
        "A*(16-neighbor shared objective experimental)": "A* 16 邻域共目标",
        "VIN-Grad": "VIN-Grad", "PPO": "PPO",
    }
    labels = {
```

源程序鉴别材料 • 末尾连续部分 30/30

```
        "length_km": "路径长度 (km)", "mean_cost": "平均相对代价",
        "sinuosity": "弯曲度", "hard_violations": "硬约束违规(点)",
        "time_s": "路径生成总耗时 (s)", "cum_relief_m": "累计高差 (m)"}
    fig, axes = plt.subplots(2, 3, figsize=(15, 9), dpi=140)
    axes = axes.ravel()
    colors = {"VIN-Grad": "#1f77ff", "PPO": "#2ca02c", "A*(baseline)": "#ff7f0e",
              "A*(weighted)": "#9467bd", "A*(turn-aware)": "#17becf",
              "A*(8-neighbor exact experimental)": "#8c564b",
              "A*(16-neighbor experimental)": "#7f7f7f",
              "A*(8-neighbor shared objective experimental)": "#bcbd22",
              "A*(16-neighbor shared objective experimental)": "#d62728",
              "Theta*(experimental)": "#e377c2"}
    for i, k in enumerate(metrics_keys):
        ax = axes[i]
        vals = [means[m][k] for m in methods]
        positions = list(range(len(methods)))
        for position, method, value in zip(positions, methods, vals):
            if value is None:
                ax.text(position, 0.02, "N/A", transform=ax.get_xaxis_transform(),
                        ha="center", va="bottom", fontsize=9)
            else:
                bar = ax.bar(position, value, color=colors[method])[0]
                ax.text(bar.get_x() + bar.get_width() / 2, value,
                        f"{value:.2f}", ha="center", va="bottom", fontsize=8)
        ax.set_title(labels[k])
        ax.set_ylabel(labels[k])
        tick_labels = [
            f"{short_names[m]}\nN={len(agg[m][k])}"
            if means[m][k] is not None else f"{short_names[m]}\nN/A"
            for m in methods
        ]
        ax.set_xticks(positions, tick_labels, fontsize=8)
        ax.margins(y=0.16)
    fig.suptitle("多算法路径规划量化对比（各走廊均值）", fontsize=14)
    fig.tight_layout(rect=[0, 0, 1, 0.96])
    try:
        fig.savefig(out_path, dpi=140)
    except Exception:
        import time as _time
        try:
            _time.sleep(0.2)
            _alt = out_path.with_name(out_path.stem + "_1" + out_path.suffix)
            fig.savefig(_alt, dpi=140)
            out_path = _alt
        except Exception as _se:
            plt.close(fig)
            print(f"  [WARN] 指标对比图保存失败(已跳过): {_se}")
            return
    plt.close(fig)
    print(f"  指标对比图保存: {out_path}")
```